Appendix C: SDK release history

Release 1.0.0 (20 January 2021)

- Initial release

Release 1.0.1 (01 February 2021)

- add `pico_get_unique_id` method to return a unique identifier for a Pico board using the identifier of the external flash
- exposed all 4 pacing timers on the DMA peripheral (previously only 2 were exposed)
- fixed ninja build (i.e. `cmake -G ninja .. ; ninja`)
- minor other improvements and bug fixes

Boot Stage 2

Additionally, a low level change was made to the way flash binaries start executing after `boot_stage2`. This was at the request of folks implementing other language runtimes. It is not generally of concern to end users, however it did require a change to the linker scripts so if you have cloned those to make modifications then you need to port across the relevant changes. If you are porting a different language runtime using the SDK boot_stage2 implementations then you should be aware that you should now have a vector table (rather than executable code) - at `0x10000100`.

Release 1.1.0 (05 March 2021)

- Added board headers for Adafruit, Pimoroni & SparkFun boards
  - new values for `PICO_BOARD` are `adafruit_feather_rp2040`, `adafruit_itsybitsy_rp2040`, `adafruit_qtpy_rp2040`, `pimoroni_keybow2040`, `pimoroni_picosystem`, `pimoroni_tiny2040`, `sparkfun_micromod`, `sparkfun_promicro`, `sparkfun_thingplus`, in addition to the existing `pico` and `vgaboard`.
  - Added additional definitions for a default SPI, I2C pins as well as the existing ones for UART
  - Allow *default* pins to be undefined (not all boards have UART for example), and SDK will compile but warn as needed in the absence of default.
  - Added additional definition for a default WS2812 compatible pin (currently unused).
- New reset options
  - Added `pico_bootsel_via_double_reset` library to allow reset to `BOOTSEL` mode via double press of a `RESET` button
  - When using `pico_stdio_usb` i.e. `stdio` connected via USB CDC to host, setting baud rate to 1200 (by default) can optionally be used to reset into `BOOTSEL` mode.
  - When using `pico-stdio_usb` i.e. `stdio` connected via USB CDC to host, an additional interface may be added to give `picotool` control over resetting the device.
- Build improvement for non-SDK or existing library builds
  - Removed additional compiler warnings (register headers now use `_u(x)` macro for unsigned values though).
  - Made build more clang friendly.

This release also contains many bug fixes, documentation updates and minor improvements.

Backwards incompatibility

There are some nominally backwards incompatible changes not worthy of a major version bump:

- `PICO_DEFAULT_UART_` defines now default to undefined if there is no default rather than `-1` previously
- The broken `multicore_sleep_core1()` API has been removed; `multicore_reset_core1` is already available to put core 1 into a deep sleep.

Release 1.1.1 (01 April 2021)

This fixes a number of bugs, and additionally adds support for a board configuration header to choose the `boot_stage2`.

Introduced absolutely no jokes at all.

Release 1.1.2 (07 April 2021)

Fixes issues with `boot_stage2` selection

Release 1.2.0 (03 June 2021)

This release contains numerous bug fixes and documentation improvements. Additionally it contains the following improvements/notable changes:

CAUTION

The `lib/tinyusb` submodule has been updated from 0.8.0 and now tracks upstream https://github.com/hathach/tinyusb.git. It is worth making sure you do a

```
$ git submodule sync
$ git submodule update
```

to make sure you are correctly tracking upstream TinyUSB if you are not checking out a clean `pico-sdk` repository.

Moving from TinyUSB 0.8.0 to TinyUSB 0.10.1 may require some minor changes to your USB code.

New/improved Board headers

- New board headers support for PICO_BOARDs `arduino_nano_rp240_connect`, `pimoroni_picolipo_4mb` and `pimoroni_picolipo_16mb`
- Missing/new `#defines` for default SPI and I2C pins have been added

Updated TinyUSB to 0.10.1

The `lib/tinyusb` submodule has been updated from 0.8.0 and now tracks upstream https://github.com/hathach/tinyusb.git

Added CMSIS core headers

CMSIS core headers (e.g. `core_cm0plus.h` and `RP2040.h`) are made available via `cmsis_core` INTERFACE library. Additionally, CMSIS standard exception naming is available via `PICO_CMSIS_RENAME_EXCEPTIONS=1`

API improvements

`pico_sync`

- Added support for recursive mutexes via `recursive_mutex_init()` and `auto_init_recursive_mutex()`
- Added `mutex_enter_timeout_us()`
- Added `critical_section_deinit()`
- Added `sem_acquire_timeout_ms()` and `sem_acquire_block_until()`

`hardware_adc`

- Added `adc_get_selected_input()`

`hardware_clocks`

- `clock_get_hz()` now returns actual achieved frequency rather than desired frequency

`hardware_dma`

- Added `dma_channel_is_claimed()`
- Added new methods for configuring/acknowledging DMA IRQs. `dma_irqn_set_channel_enabled()`, `dma_irqn_set_channel_mask_enabled()`, `dma_irqn_get_channel_status()`, `dma_irqn_acknowledge_channel()` etc.

`hardware_exception`

New library for setting ARM exception handlers:

- Added `exception_set_exclusive_handler()`, `exception_restore_handler()`, `exception_get_vtable_handler()`

`hardware_flash`

- Exposed previously private function `flash_do_cmd()` for low level flash command execution

`hardware_gpio`

- Added `gpio_set_input_hysteresis_enabled()`, `gpio_is_input_hysteresis_enabled()`, `gpio_set_slew_rate()`, `gpio_get_slew_rate()`, `gpio_set_drive_strength()`, `gpio_get_drive_strength()`, `gpio_get_out_level()`, `gpio_set_irqover()`

`hardware_i2c`

- Corrected a number of incorrect hardware register definitions
- A number of edge cases in the i2c code fixed

hardware_interp

- Added `interp_lane_is_claimed()`, `interp_unclaim_lane_mask()`

hardware_irq

- Notably fixed the `PICO_LOWEST/HIGHEST_IRQ_PRIORITY` values which were backwards!

hardware_pio

- Added new methods for configuring/acknowledging PIO interrupts (`pio_set_irqn_source_enabled()`, `pio_set_irqn_source_mask_enabled()`, `pio_interrupt_get()`, `pio_interrupt_clear()` etc.)
- Added `pio_sm_is_claimed()`

hardware_spi

- Added `spi_get_baudrate()`
- Changed `spi_init()` to return the set/achieved baud rate rather than void
- Changed `spi_is_writable()` to return bool not size_t (it was always 1/0)

hardware_sync

- Notable documentation improvements for spin lock functions
- Added `spin_lock_is_claimed()`

hardware_timer

- Added `busy_wait_ms()` to match `busy_wait_us()`
- Added `hardware_alarm_is_claimed()`

pico_float/pico_double

- Correctly save/restore divider state if floating point is used from interrupts

pico_int64_ops

- Added `PICO_INT64_OPS_IN_RAM` flag to move code into RAM to avoid veneers when calling code is in RAM

pico_runtime

- Added ability to override panic function by setting `PICO_PANIC_FUNCTION=foo` to then use `foo` as the implementation, or setting `PICO_PANIC_FUNCITON=` to simply breakpoint, saving some code space

pico_unique_id

- Added `pico_get_unique_board_id_string()`.

General code improvements

- Removed additional classes of compiler warnings
- Added some missing `const` to method parameters

SVD

- USB DPRAM for device mode is now included

pioasm

- Added `#pragma once` to C/C++ output

RTOS interoperability

Improvements designed to make porting RTOSes either based on the SDK or supporting SDK code easier.

- Added `PICO_DIVIDER_DISABLE_INTERRUPTS` flag to optionally configure all uses of the hardware divider to be guarded by disabling interrupts, rather than requiring on the RTOS to save/restore the divider state on context switch
- Added new abstractions to `pico/lock_core.h` to allow an RTOS to inject replacement code for SDK based low level wait, notify and sleep/timeouts used by synchronization primitives in `pico_sync` and for `sleep_` methods. If an RTOS implements these few simple methods, then all SDK semaphore, mutex, queue, sleep methods can be safely used both within/to/from RTOS tasks, but also to communicate with non-RTOS task aware code, whether it be existing libraries and IRQ handlers or code running perhaps (though not necessarily) on the other core

CMake build changes

Substantive changes have been made to the CMake build, so if you are using a hand crafted non-CMake build, you **will** need to update your compile/link flags. Additionally changed some possibly confusing status messages from CMake build generation to be debug only

Boot Stage 2

- New boot stage 2 for `AT25SF128A`

Release 1.3.0 (02 November 2021)

This release contains numerous bug fixes and documentation improvements. Additionally, it contains the following notable changes/improvements:

Updated TinyUSB to 0.12.0

- The `lib/tinyusb` submodule has been updated from 0.10.1 to 0.12.0. See https://github.com/hathach/tinyusb/releases/tag/0.11.0 and https://github.com/hathach/tinyusb/releases/tag/0.12.0 for release notes.
- Improvements have been made for projects that include TinyUSB and also compile with enhanced warning levels and `-Werror`. Warnings have been fixed in RP2040 specific TinyUSB code, and in TinyUSB headers, and a new cmake function `suppress_tinyusb_warnings()` has been added, that you may call from your `CMakeLists.txt` to suppress warnings in other TinyUSB C files.

New Board Support

The following boards have been added and may be specified via `PICO_BOARD`:

- `adafruit_trinkey_qt2040`
- `melopero_shake_rp2040`
- `pimoroni_interstate75`
- `pimoroni_plasma2040`
- `pybstick26_rp2040`
- `waveshare_rp2040_lcd_0.96`
- `waveshare_rp2040_plus_4mb`
- `waveshare_rp2040_plus_16mb`
- `waveshare_rp2040_zero`

Updated SVD, `hardware_regs`, `hardware_structs`

The RP2040 SVD has been updated, fixing some register access types and adding new documentation.

The `hardware_regs` headers have been updated accordingly.

The `hardware_structs` headers which were previously hand coded, are now generated from the SVD, and retain select documentation from the SVD, including register descriptions and register bit-field tables.

e.g. what was once

```
typedef struct {
    io_rw_32 ctrl;
    io_ro_32 fstat;
    ...
```

becomes:

```
// Reference to datasheet: https://datasheets.raspberrypi.com/rp2040/rp2040-datasheet.pdf#tab-
registerlist_pio
//
// The _REG_ macro is intended to help make the register navigable in your IDE (for example, using
the "Go to Definition" feature)
// _REG_(x) will link to the corresponding register in hardware/regs/pio.h.
//
// Bit-field descriptions are of the form:
// BITMASK [BITRANGE]: FIELDNAME (RESETVALUE): DESCRIPTION

typedef struct {
    _REG_(PIO_CTRL_OFFSET) // PIO_CTRL
    // PIO control register
    // 0x00000f00 [11:8]  : CLKDIV_RESTART (0): Restart a state machine's clock divider from an
initial phase of 0
    // 0x000000f0 [7:4]   : SM_RESTART (0): Write 1 to instantly clear internal SM state which may
be otherwise difficult...
    // 0x0000000f [3:0]   : SM_ENABLE (0): Enable/disable each of the four state machines by
writing 1/0 to each of these four bits
    io_rw_32 ctrl;

    _REG_(PIO_FSTAT_OFFSET) // PIO_FSTAT
```

```
    // FIFO status register
    // 0x0f000000 [27:24] : TXEMPTY (0xf): State machine TX FIFO is empty
    // 0x000f0000 [19:16] : TXFULL (0): State machine TX FIFO is full
    // 0x00000f00 [11:8]  : RXEMPTY (0xf): State machine RX FIFO is empty
    // 0x0000000f [3:0]   : RXFULL (0): State machine RX FIFO is full
    io_ro_32 fstat;
    ...
```

Behavioural Changes

There were some behavioural changes in this release:

pico_sync

SDK 1.2.0 previously added recursive mutex support using the existing (previously non-recursive) `mutex_` functions. This caused a performance regression, and the only clean way to fix the problem was to return the `mutex_` functions to their pre-SDK 1.2.0 behaviour, and split the recursive mutex functionality out into separate `recursive_mutex_` functions with a separate `recursive_mutex_` type.

Code using the SDK 1.2.0 recursive mutex functionality will need to be changed to use the new type and functions, however as a convenience, the pre-processor define `PICO_MUTEX_ENABLE_SDK120_COMPATIBILITY` may be set to 1 to retain the SDK 1.2.0 behaviour at the cost of an additional performance penalty. The ability to use this pre-processor define will be removed in a subsequent SDK version.

pico_platform

- `pico.h` and its dependencies have been slightly refactored so it can be included by assembler code as well as C/C code. This ensures that assembler code and C/C code follow the same board configuration/override order and see the same configuration defines. This should not break any existing code, but is notable enough to mention.
- `pico/platform.h` is now fully documented.

pico_standard_link

`-Wl,max-page-size=4096` is now passed to the linker, which is beneficial to certain users and should have no discernible impact on the rest.

Other Notable Improvements

hardware_base

- Added `xip_noalloc_alias(addr)`, `xip_nocache_alias(addr)`, `xip_nocache_noalloc_alias(addr)` macros for converting a flash address between XIP aliases (similar to the `hw_xxx_alias(addr)` macros).

hardware_dma

- Added `dma_timer_claim()`, `dma_timer_unclaim()`, `dma_claim_unused_timer()` and `dma_timer_is_claimed()` to manage ownership of DMA timers.
- Added `dma_timer_set_fraction()` and `dma_get_timer_dreq()` to facilitate pacing DMA transfers using DMA timers.

hardware_i2c

- Added `i2c_get_dreq()` function to facilitate configuring DMA transfers to/from an I2C instance.

hardware_irq

- Added `irq_get_priority()`.
- Fixed implementation when `PICO_DISABLE_SHARED_IRQ_HANDLERS=1` is specified, and allowed `irq_add_shared_handler` to be used in this case (as long as there is only one handler - i.e. it behaves exactly like `irq_set_exclusive_handler`).
- Sped up IRQ priority initialization which was slowing down per core initialization.

hardware_pio

- `pio_encode_` functions in `hardware/pico_instructions.h` are now documented.

hardware_pwm

- Added `pwm_get_dreq()` function to facilitate configuring DMA transfers to a PWM slice.

hardware_spi

- Added `spi_get_dreq()` function to facilitate configuring DMA transfers to/from an SPI instance.

hardware_uart

- Added `uart_get_dreq()` function to facilitate configuring DMA transfers to/from a UART instance.

hardware_watchdog

- Added `watchdog_enable_caused_reboot()` to distinguish a watchdog reboot caused by a watchdog timeout after calling `watchdog_enable()` from other watchdog reboots (e.g. that are performed when a UF2 is dragged onto a device in BOOTSEL mode).

pico_bootrom

- Added new constants and function signature typedefs to `pico/bootrom.h` to facilitate calling bootrom functions directly.

pico_multicore

- Improved documentation in `pico/multicore.h`; particularly, `multicore_lockout_` functions are newly documented.

pico_platform

- `PICO_RP2040` is now defined to 1 in `PICO_PLATFORM=rp2040` (i.e. normal) builds.

pico_stdio

- Added `puts_raw()` and `putchar_raw()` to skip CR/LF translation if enabled.

- Added `stdio_usb_connected()` to detect CDC connection when using `stdio_usb`.
- Added `PICO_STDIO_USB_CONNECT_WAIT_TIMEOUT_MS` define that can be set to wait for a CDC connection to be established during initialization of `stdio_usb`. Note: value -1 means indefinite. This can be used to prevent initial program output being lost, at the cost of requiring an active CDC connection.
- Fixed `semihosting_putc` which was completely broken.

`pico_usb_reset_interface`

- This new library contains `pico/usb_reset_interface.h` split out from `stdio_usb` to facilitate inclusion in external projects.

CMake build

- `OUTPUT_NAME` target property is now respected when generating supplemental files (`.BIN`, `.HEX`, `.MAP`, `.UF2`)

pioasm

- Operator precedence of `*`, `/`, `-`, `+` have been fixed
- Incorrect MicroPython output has been fixed.

`elf2uf2`

- A bug causing an error with binaries produced by certain other languages has been fixed.

Release 1.3.1 (18 May 2022)

This release contains numerous bug fixes and documentation improvements which are not all listed here; you can see the full list of individual commits here.

New Board Support

The following boards have been added and may be specified via `PICO_BOARD`:

- `adafruit_kb2040`
- `adafruit_macropad_rp2040`
- `eetree_gamekit_rp2040`
- `garatronic_pybstick26_rp2040` (renamed from `pybstick26_rp2040`)
- `pimoroni_badger2040`
- `pimoroni_motor2040`
- `pimoroni_servo2040`
- `pimoroni_tiny2040_2mb`
- `seeed_xiao_rp2040`
- `solderparty_rp2040_stamp_carrier`
- `solderparty_rp2040_stamp`

- wiznet_w5100s_evb_pico

Notable Library Changes/Improvements

hardware_dma

- New documentation has been added to the dma_channel_abort() function describing errata RP2040-E13, and how to work around it.

hardware_irq

- Fixed a bug related to removing and then re-adding shared IRQ handlers. It is now possible to add/remove handlers as documented.
- Added new documentation clarifying the fact the shared IRQ handler ordering "priorities" have values that increase with higher priority vs. Cortex M0+ IRQ priorites which have values that decrease with priority!

hardware_pwm

- Added a pwm_config_set_clkdiv_int_frac() method to complement pwm_config_set_clkdiv_int() and pwm_config_set_clkdiv().

hardware_pio

- Fixed the pio_set_irqn_source_mask_enabled() method which previously affected the wrong IRQ.

hardware_rtc

- Added clarification to rtc_set_datetime() documentation that the new value may not be visible to a rtc_get_datetime() very soon after, due to crossing of clock domains.

pico_platform

- Added a busy_wait_at_least_cycles() method as a convenience method for a short tight-loop counter-based delay.

pico_stdio

- Fixed a bug related to removing stdio "drivers". stdio_set_driver_enabled() can now be used freely to dynamically enable and disable drivers during runtime.

pico_time

- Added an is_at_the_end_of_time() method to check if a given time matches the SDK's maximum time value.

Runtime

A bug in __ctzdi2() aka __builtin_ctz(uint64_t) was fixed.

Build

- Compilation with GCC 11 is now supported.
- `PIOASM_EXTRA_SOURCE_FILES` is now actually respected.

pioasm

- Input files with Windows (CRLF) line endings are now accepted.
- A bug in the python output was fixed.

`elf2uf2`

- Extra padding was added to the UF2 output of misaligned or non-contiguous binaries to work around errata RP2040-E14.

NOTE

The 1.3.0 release of the SDK incorrectly squashed the history of the changes. A new merge commit has been added to restore the full history, and the 1.3.0 tag has been updated

Release 1.4.0 (30 June 2022)

This release adds wireless support for the Raspberry Pi Pico W, adds support for other new boards, and contains various bug fixes, documentation improvements, and minor improvements/added functionality. You can see the full list of individual commits here.

New Board Support

The following boards have been added and may be specified via `PICO_BOARD`:

- `pico_w`
- `datanoisetv_rp2040_dsp`
- `solderparty_rp2040_stamp_round_carrier`

Wireless Support

- Support for the Raspberry Pi Pico W is now included with the SDK (`PICO_BOARD=pico_w`). The Pico W uses a driver for the wireless chip called `cyw43_driver` which is included as a submodule of the SDK. You need to initialize this submodule for Pico W wireless support to be available. Note that the LED on the Pico W board is only accessible via the wireless chip, and can be accessed via `cyw43_arch_gpio_put()` and `cyw43_arch_gpio_get()` (part of the `pico_cyw43_arch` library described below). As a result of the LED being on the wireless chip, there is no `PICO_DEFAULT_LED_PIN` setting and the default LED based examples in pico-examples do not work with the Pico W.
- IP support is provided by lwIP which is also included as a submodule which you should initialize if you want to use it.

  The following libraries exposing lwIP functionality are provided by the SDK:

  - `pico_lwip_core` (included in `pico_lwip`)
  - `pico_lwip_core4` (included in `pico_lwip`)

- pico_lwip_core6 (included in pico_lwip)
- pico_lwip_netif (included in pico_lwip)
- pico_lwip_sixlowpan (included in pico_lwip)
- pico_lwip_ppp (included in pico_lwip)
- pico_lwip_api (this is a blocking API that may be used with FreeRTOS and is not included in pico_lwip)

As referenced above, the SDK provides a pico_lwip which aggregates all of the commonly needed lwIP functionality. You are of course free to use the substituent libraries explicitly instead.

The following libraries are provided that contain the equivalent lwIP application support:

- pico_lwip_snmp
- pico_lwip_http
- pico_lwip_makefsdata
- pico_lwip_iperf
- pico_lwip_smtp
- pico_lwip_sntp
- pico_lwip_mdns
- pico_lwip_netbios
- pico_lwip_tftp
- pico_lwip_mbedtls

- Integration of the IP stack and the cyw43_driver network driver into the user's code is handled by pico_cyw43_arch. Both the IP stack and the driver need to do work in response to network traffic, and pico_cyw43_arch provides a variety of strategies for servicing that work. Four architecture variants are currently provided as libraries:
  - pico_cyw43_arch_lwip_poll - For using the RAW lwIP API (NO_SYS=1 mode) with polling. With this architecture the user code must periodically poll via cyw43_arch_poll() to perform background work. This architecture matches the common use of lwIP on microcontrollers, and provides no multicore safety
  - pico_cyw43_arch_lwip_threadsafe_background - For using the RAW lwIP API (NO_SYS=1 mode) with multicore safety, and automatic servicing of the cyw43_driver and lwIP in the background. User polling is not required with this architecture, but care should be taken as lwIP callbacks happen in an IRQ context.
  - pico_cyw43_arch_lwip_sys_freertos - For using the full lwIP API including blocking sockets in OS mode (NO_SYS=0), along with multicore/task safety, and automatic servicing of the cyw43_driver and the lwIP stack in a separate task. This powerful architecture works with both SMP and non-SMP variants of the RP2040 port of FreeRTOS-Kernel. Note you must set FREERTOS_KERNEL_PATH in your build to use this variant.
  - pico_cyw43_arch_none - If you do not need the TCP/IP stack but wish to use the on-board LED or other wireless chip connected GPIOs.

  See the library documentation or the pico/cyw43_arch.h header for more details.

Notable Library Changes/Improvements

hardware_dma

- Added dma_unclaim_mask() function for un-claiming multiple DMA channels at once.
- Added channel_config_set_high_priority() function to set the channel priority via a channel config object.

hardware_gpio

- Improved the documentation for the pre-existing gpio IRQ functions which use the "one callback per core" callback mechanism, and added a `gpio_set_irq_callback()` function to explicitly set the callback independently of enabling per pin GPIO IRQs.
- Reduced the latency of calling the existing "one callback per core" GPIO IRQ callback.
- Added new support for the user to add their own shared GPIO IRQ handler independent of the pre-existing "one callback per core" callback mechanism, allowing for independent usage of GPIO IRQs without having to share one handler.

  See the documentation in `hardware/irq.h` for full details of the functions added:

  - `gpio_add_raw_irq_handler()`
  - `gpio_add_raw_irq_handler_masked()`
  - `gpio_add_raw_irq_handler_with_order_priority()`
  - `gpio_add_raw_irq_handler_with_order_priority_masked()`
  - `gpio_remove_raw_irq_handler()`
  - `gpio_remove_raw_irq_handler_masked()`
- Added a `gpio_get_irq_event_mask()` utility function for use by the new "raw" IRQ handlers.

hardware_irq

- Added `user_irq_claim()`, `user_irq_unclaim()`, `user_irq_claim_unused()` and `user_irq_is_claimed()` functions for claiming ownership of the **user** IRQs (the ones numbered 26-31 and not connected to any hardware). Uses of the **user** IRQs have been updated to use these functions. For `stdio_usb`, the `PICO_STDIO_USB_LOW_PRIORITY_IRQ` define is still respected if specified, but otherwise an unclaimed one is chosen.
- Added an `irq_is_shared_handler()` function to determine if a particular IRQ uses a shared handler.

pico_sync

- Added a `sem_try_acquire()` function, for non-blocking acquisition of a semaphore.

pico_stdio

- `stderr` is now supported and goes to the same destination as `stdout`.
- Zero timeouts for `getchar_timeout_us()` are now correctly honored (previously they were a 1µs minimum).

stdio_usb

- The use of a 1ms timer to handle background TinyUSB work has been replaced with use of a more interrupt driven approach using a **user** IRQ for better performance. Note this new feature is disabled if shared IRQ handlers are disabled via `PICO_DISABLE_SHARED_IRQ_HANDLERS=1`

Miscellaneous

- `get_core_num()` has been moved to `pico/platform.h` from `hardware/sync.h`.
- The C library function `realloc()` is now multicore safe too.
- The minimum PLL frequency has been increased from 400 MHz to 750 MHz to improve stability across operating conditions. This should not affect the majority of users in any way, but may impact those trying to set particularly

low clock frequencies. If you do wish to return to the previous minimum, you can set `PICO_PLL_VCO_MIN_FREQ_MHZ` back to `400`. There is also a new `PICO_PLL_VCO_MAX_FREQ_MHZ` which defaults to `1600`.

Build

- Compilation with GCC 12 is now supported.

Release 1.5.0 (11 February 2023)

This release contains new libraries and functionality, along with numerous bug fixes and documentation improvements.

Highlights are listed below, or you can see the full list of individual commits here, and the full list of resolved issues here.

New Board Support

The following boards have been added and may be specified via `PICO_BOARD`:

- `nullbits_bit_c_pro`
- `waveshare_rp2040_lcd_1.28`
- `waveshare_rp2040_one`

Library Changes/Improvements

`hardware_clocks`

- `clock_gpio_init()` now takes a `float` for the clock divider value, rather than an `int`.
- Added `clock_gpio_init_int_frac()` function to allow initialization of integer and fractional part of the clock divider value, without using `float`.
- Added `--ref-min` option to `vcocalc.py` to override the minimum reference frequency allowed.
- `vcocalc.py` now additionally considers reference frequency dividers greater than 1.

`hardware_divider`

- Improved the performance of `hw_divider_` functions.

`hardware_dma`

- Added `dma_sniffer_set_output_invert_enabled()` and `dma_sniffer_set_output_reverse_enabled()` functions to configure the DMA sniffer.
- Added `dma_sniffer_set_data_accumulator()` and `dma_sniffer_get_data_accumulator()` functions to access the DMA sniffer accumulator.

`hardware_i2c`

- Added `i2c_get_instance()` function for consistency with other `hardware_` libraries.

- Added `i2c_read_byte_raw()`, `i2c_write_byte_raw()` functions to directly read and write the I2C data register for an I2C instance.

hardware_timer

- Added `hardware_alarm_claim_unused()` function to claim an unused hardware timer.

pico_cyw43_arch

- Added `cyw43_arch_wifi_connect_bssid_` variants of `cyw43_arch_wifi_connect_` functions to allow connection to a specific access point.
- Blocking `cyw43_arch_wifi_connect_` functions now continue trying to connect rather than failing immediately if the network is not found.
- `cyw43_arch_wifi_connect_` functions now return consistent return codes (`PICO_OK`, or `PICO_ERROR_XXX`).
- The `pico_cyw43_arch` library has been completely rewritten on top of the new `pico_async_context` library that generically abstracts the different types of asynchronous operation (`poll`, `threadsafe_background` and `freertos`) previously handled in a bespoke fashion by `pico_cyw43_arch`. Many edge case bugs have been fixed as a result of this. Note that this change should be entirely backwards compatible from the user point of view.
- `cyw43_arch_init()` and `cyw43_arch_deinit()` functions are now very thin layers which handle `async_context` life-cycles, along with adding support for the `cyw43_driver`, lwIP, BTstack etc. to that `async_context`. Currently, these mechanisms remain the preferred documented way to initialize Pico W networking, however you are free to do similar initialization/de-initialization yourself.
- Added `cyw43_arch_set_async_context()` function to specify a custom `async_context` prior to calling `cyw43_arch_init*()`
- Added `cyw43_arch_async_context()` function to get the `async_context` used by the CYW43 architecture support.
- Added `cyw43_arch_init_default_async_context()` function to return the `async_context` that `cyw43_arch_init*()` would initialize if one has not been set by the user.
- Added `cyw43_arch_wait_for_work_until()` function to block until there is networking work to be done. This is most useful for `poll` style applications that have no other work to do and wish to sleep until `cyw43_arch_poll()` needs to be called again.

pico_cyw43_driver

- The functionality has been clarified into 3 separate libraries:
  - `cyw43_driver` - the raw cyw43_driver code.
  - `cyw43_driver_picow` - additional support for communication with the Wi-Fi chip over SPI on Pico W.
  - `pico_cyw43_driver` - integration of the cyw43_driver with the `pico-sdk` via `async_context`
- Added `CYW43_WIFI_NVRAM_INCLUDE_FILE` define to allow user to override the NVRAM file.

pico_divider

- Improved the performance of 64-bit divider functions.

pico_platform

- Add `panic_compact()` function that discards the message to save space in non-debug (`NEBUG` defined) builds.

pico_runtime

- Added proper implementation of certain missing newlib system APIs: _gettimeofday(), _times(), _isatty(), _getpid().
- The above changes enable certain additional C/C++ library functionality such as gettimeofday(), times() and std::chrono.
- Added settimeofday() implementation such that gettimeofday() can be meaningfully used.
- Added default (return -1) implementations of the remaining newlib system APIs: _open(), _close(), _lseek(), _fstat(), _isatty(), _kill(), to prevent warnings on GCC 12.
- Made all newlib system API implementations *weak* so the user can override them.

pico_stdio

- pico_stdio allows for outputting from within an IRQ handler that creates the potential for deadlocks (especially with pico_stdio_usb), and the intention is to *not* deadlock but instead discard output in any cases where a deadlock would otherwise occur. The code has been revamped to avoid more deadlock cases, and a new define PICO_STDIO_DEADLOCK_TIMEOUT_MS has been added to catch remaining cases that might be caused by user level locking.
- Added stdio_set_chars_available_callback() function to set a callback to be called when input is available. See also the new PICO_STDIO_USB_SUPPORT_CHARS_AVAILABLE_CALLBACK and PICO_STDIO_UART_SUPPORT_CHARS_AVAILABLE_CALLBACK defines which both default to 1 and control the availability of this new feature for USB and UART stdio respectively (at the cost of a little more code).
- Improved performance of stdio_semihosting.
- Give the user more control over the USB descriptors of stdio_usb via USBD_VID, USBD_PID, USBD_PRODUCT, PICO_STDIO_USB_CONNECTION_WITHOUT_DTR and PICO_STDIO_USB_DEVICE_SELF_POWERED

pico_sync

- Added critical_section_is_initialized() function to test if a critical section has been initialized.
- Added mutex_try_enter_block_until() function to wait only up to a certain time to acquire a mutex.

pico_time

- Added from_us_since_boot() function to convert a uint64_t timestamp to an absolute_time_t.
- Added absolute_time_min() function to return the earlier of two absolute_time_t values.
- Added alarm_pool_create_with_unused_hardware_alarm() function to create an alarm pool using a hardware alarm number claimed using hardware_alarm_claim().
- Added alarm_pool_core_num() function to determine what core an alarm pool runs on.
- Added alarm_pool_add_alarm_at_force_in_context() function to add an alarm, and have it always run in the IRQ context even if the target time is in the past, or during the call. This may be simpler in some cases than dealing with the fire_if_past parameters to existing functions, and avoids some callbacks happening from non-IRQ context.

pico_lwip

- Added pico_lwip_mqtt library to expose the MQTT app functionality in lwIP.
- Added pico_lwip_mdns library to expose the MDNS app functionality in lwIP.
- Added pico_lwip_freertos library for NO_SYS=0 with FreeRTOS as a complement to pico_lwip_nosys for NO_SYS=1.

TinyUSB

- TinyUSB has upgraded from 0.12.0 to 0.15.0. See TinyUSB release notes here for details.
- Particularly *host* support should be massively improved.
- Defaulted new TinyUSB `dcd_rp2040` driver's `TUD_OPT_RP2040_USB_DEVICE_UFRAME_FIX` variable to `1` as a workaround for errata RP2040-E15. This fix is required for correctness, but comes at the cost of some performance, so applications that won't ever be plugged into a Pi 4 or Pi 400 can optionally disable this by setting the value of `TUD_OPT_RP2040_USB_DEVICE_UFRAME_FIX` to `0` either via `target_compile_definitions` in their `CMakeLists.txt` or in their `tusb_config.h`.

New Libraries

`pico_async_context`

- Provides support for asynchronous events (timers/IRQ notifications) to be handled in a safe context without concurrent execution (as required by many asynchronous 3rd party libraries).
- Provides implementations matching those previously implemented in `pico_cyw43_arch`:
  - `poll` - Not thread-safe; the user must call `async_context_poll()` periodically from their main loop, but can call `async_context_wait_for_work_until()` to block until work is required.
  - `threadsafe_background` - No polling is required; instead asynchronous work is performed in a low priority IRQ. Locking is provided such that IRQ/non-IRQ or multiple cores can interact safely.
  - `freertos` - Asynchronous work is performed in a separate FreeRTOS task.
- `async_context` guarantees all callbacks happen on a single core.
- `async_context` supports multiple instances for providing independent context which can execute concurrently with respect to each other.

`pico_i2c_slave`

- A (slightly modified) pico_i2c_slave library from https://github.com/vmilea/pico_i2c_slave
- Adds a callback style event API for handling I2C slave requests.

`pico_mbedtls`

- Added `pico_mbedtls` library to provide MBed TLS support. You can depend on both `pico_lwip_mbedtls` and `pico_mbedtls` to use MBed TLS and lwIP together. See the tls_client example in `pico-examples` for more details.

`pico_rand`

- Implements a new Random Number Generator API.
- `pico_rand` generates random numbers at runtime utilizing a number of possible entropy sources, and uses those sources to modify the state of a 128-bit 'Pseudo Random Number Generator' implemented in software.
- Adds `get_rand_32()`, `get_rand_64()` and `get_rand_128()` functions to return largely unpredictable random numbers (which should be different on each board/run for example).

Miscellaneous

- Added a new header `hardware/structs/nvic.h` with a struct for the Arm Cortex M0+ NVIC available via the `nvic_hw` pointer.
- Added new `PICO_CXX_DISABLE_ALLOCATION_OVERRIDES` which can be set to `1` if you do not want `pico_standard_link` to include non-exceptional overrides of `std::new`, `std::new[]`, `std::delete` and `std::delete[]` when exceptions are disabled.
- `elf2uf2` now correctly uses `LMA` instead of `VMA` of the entry point to determine binary type (flash/RAM). This is required to support some exotic binaries correctly.

Build

- The build will now check for a functional compiler via the standard `CMake` mechanism.
- The build will pick up pre-installed `elf2uf2` and `pioasm` if found via an installed `pico-sdk-tools` `CMake` package. If it can do so, then no native compiler is required for the build!
- It is now possible to switch the board type `PICO_BOARD` in an existing `CMake` build directory.
- `ARCHIVE_OUTPUT_DIRECTORY` is now respected in build for `UF2` output files.
- Spaces are now supported in the path to the `pico-sdk`
- All libraries `xxx` in the `pico-sdk` now support a `xxx_headers` variant that just pulls in the libraries' headers. These `xxx_headers` libraries correctly mirror the dependencies of the `xxx` libraries, so you can use `xxx_headers` instead of `xxx` as your dependency if you do not want to pull in any implementation files (perhaps if you are making a `STATIC` library). Actually the "all" is not quite true, non-code libraries such as `pico_standard_link` and `pico_cxx_options` are an exception.

Bluetooth Support for Pico W (BETA)

The support is currently available as a beta. More details will be forthcoming with the actual release. In the meantime, there are examples in pico-examples.

Key changes:

- The Bluetooth API is provided by BTstack.
- The following new libraries are provided that expose core BTstack functionality:
  - `pico_btstack_ble` - Adds Bluetooth Low Energy (LE) support.
  - `pico_btstack_classic` - Adds Bluetooth Classic support.
  - `pico_btstack_sbc_encoder` - Adds Bluetooth Sub Band Coding (SBC) encoder support.
  - `pico_btstack_sbc_decoder` - Adds Bluetooth Sub Band Coding (SBC) decoder support.
  - `pico_btstack_bnep_lwip` - Adds Bluetooth Network Encapsulation Protocol (BNEP) support using LwIP.
  - `pico_btstack_bnep_lwip_sys_freertos` - Adds Bluetooth Network Encapsulation Protocol (BNEP) support using LwIP with FreeRTOS for `NO_SYS=0`.
- The following integration libraries are also provided:
  - `pico_btstack_run_loop_async_context` - provides a common `async_context` backed implementation of a BTstack *"run loop"* that can be used for all BTstack use with the `pico-sdk`.
  - `pico_btstack_flash_bank` - provides a sample implementation for storing required Bluetooth state in flash.
  - `pico_btstack_cyw43` - integrates BTstack with the CYW43 driver.
- Added `CMake` function `pico_btstack_make_gatt_header` that can be used to run the BTstack `compile_gatt` tool to make a

GATT header file from a BTstack `GATT` file.

- Updated `pico_cyw43_driver` and `cyw43_driver` to support HCI communication for Bluetooth.
- Updated `cyw43_driver_picow` to support Pico W specific HCI communication for Bluetooth over SPI.
- Updated `cyw43_arch_init()` and `cyw43_arch_deinit()` to additionally handle Bluetooth support if `CYW43_ENABLE_BLUETOOTH` is `1` (as it will be automatically if you depend on `pico_btstack_cyw43`).

Release 1.5.1 (14 June 2023)

This release is largely a bug fix release, however it also makes Bluetooth support official and adds some new libraries and functionality.

Highlights are listed below, or you can see the full list of individual commits here, and the full list of resolved issues here.

Board Support

The following board has been added and may be specified via `PICO_BOARD`:

- `pololu_3pi_2040_robot`

The following board configurations have been modified:

- `adafruit_itsybitsy_rp2040` - corrected the mismatched `PICO_DEFAULT_I2C` bus number (favors the breadboard pins not the stemma connector).
- `sparkfun_thingplus` - added WS2812 pin config.

Library Changes/Improvements

hardware_dma

- Added `dma_channel_cleanup()` function that can be used to clean up a dynamically claimed DMA channel after use, such that it won't be in a surprising state for the next user, making sure that any in-flight transfer is aborted, and no interrupts are left pending.

hardware_spi

- The `spi_set_format`, `spi_set_slave`, `spi_set_baudrate` functions that modify the configuration of an SPI instance, now disable the SPI while changing the configuration as specified in the data sheet.

pico_async_context

- Added `user_data` member to `async_when_pending_worker_t` to match `async_at_time_worker_t`.

pico_cyw43_arch

- Added `cyw43_arch_disable_sta_mode()` function to complement `cyw43_arch_enable_sta_mode()`.
- Added `cyw43_arch_disable_ap_mode()` function to complement `cyw43_arch_enable_ap_mode()`.

pico_stdio_usb

- The 20-character limit for descriptor strings USBD_PRODUCT and USBD_MANUFACTURER can now be extended by defining USBD_DESC_STR_MAX.
- PICO_STDIO_USB_CONNECT_WAIT_TIMEOUT_MS is now supported in the build as well as compiler definitions; if it is set in the build, it is added to the compile definitions.

pico_rand

- Fixed poor randomness when PICO_RAND_ENTROPY_SRC_BUS_PERF_COUNTER=1.

PLL and Clocks

- The set_sys_clock_pll and set_sys_clock_khz methods now reference a pre-processor define PICO_CLOCK_AJDUST_PERI_CLOCK_WITH_SYS_CLOCK. If set to 1, the peripheral clock is updated to match the new system clock, otherwise the preexisting behavior (of setting the peripheral clock to a safe 48 MHz) is preserved.
- Support for non-standard crystal frequencies, and compile-time custom clock configurations:
  - The new define XOSC_KHZ is used in preference to the preexisting XOSC_MHZ to define the crystal oscillator frequency. This value is now also correctly plumbed through the various clock setup functions, such that they behave correctly with a crystal frequency other than 12 MHz. XOSC_MHZ will be automatically defined for backwards compatibility if XOSC_KHZ is an exact multiple of 1000 KHz. Note that either XOSC_MHZ or XOSC_KHZ may be specified by the user, but not both.
  - The new define PLL_COMMON_REFDIV can be specified to override the default reference divider of 1.
  - The new defines PLL_SYS_VCO_FREQ_KHZ, PLL_SYS_POSTDIV1 and PLL_SYS_POSTDIV2 are used to configure the system clock PLL during runtime initialization. These are defaulted for you if SYS_CLK_KHZ=125000, XOSC_KHZ=12000 and PLL_COMMON_REFDIV=1. You can modify these values in your CMakeLists.txt if you want to configure a different system clock during runtime initialization, or are using a non-standard crystal.
  - The new defines PLL_USB_VCO_FREQ_KHZ, PLL_USB_POSTDIV1 and PLL_USB_POSTDIV2 are used to configure the USB clock PLL during runtime initialization. These are defaulted for you if USB_CLK_KHZ=48000, XOSC_KHZ=12000 and PLL_COMMON_REFDIV=1. You can modify these values in your CMakeLists.txt if you want to configure a different USB clock if you are using a non-standard crystal.
  - The new define PICO_PLL_VCO_MIN_FREQ_KHZ is used in preference to the pre-existing PICO_PLL_VCO_MIN_FREQ_MHZ, though specifying either is supported.
  - The new define PICO_PLL_VCO_MAX_FREQ_KHZ is used in preference to the pre-existing PICO_PLL_VCO_MAX_FREQ_MHZ, though specifying either is supported.

New Libraries

pico_flash

- This is a new higher level library than hardware_flash. It provides helper functions to facilitate getting into a state where it is safe to write to flash (the default implementation disables interrupts on the current core, and if necessary, makes sure the other core is running from RAM, and has interrupts disabled).
- Adds a flash_safe_execute() function to execute a callback function while in the "safe" state.
- Adds a flash_safe_execute_core_init() function which must be called from the "other core" when using pico_multicore to enable the cooperative support for entering a "safe" state.
- Supports user override of the mechanism by overriding the get_flash_safety_helper() function.

Miscellaneous

- All assembly (including inline) in the SDK now uses the `unified` syntax.
  - New C macros `pico_default_asm( ⋯ )` and `pico_default_asm_volatile( ⋯ )` are provided that are equivalent to `asm andasm volatile` blocks, but with a `.syntax unified` at the beginning.
- A new assembler macro `pico_default_asm_setup` is provided to configure the correct CPU and dialect.
- Some code cleanup to make the SDK code at least compile cleanly on Clang and IAR.

Build

- `PICO_BOARD` and `PICO_BOARD_HEADER_DIRS` now correctly use the latest environment variable value if present.
- A CMake performance regression due to repeated calls to `find_package` has been fixed.
- Experimental support is provided for compiling with Clang. As an example, you can build with the LLVM Embedded Toolchain for Arm, noting however that currently only version 14.0.0 works, as later versions use `picolib` rather than `newlib`.
  - Note that if you are using TinyUSB you need to use the latest master to compile with Clang.

```
$ mkdir clang_build
$ cd clang_build
$ cmake -DPICO_COMPILER=pico_arm_clang -DPICO_TOOLCHAIN_PATH=/path/to/arm-embedded-llvm
-14.0.0 ..
$ make
```

Bluetooth Support for Pico W

The support is now official. Please find examples in pico-examples.

- The Bluetooth API is provided by BTstack.
- The following libraries are provided that expose core BTstack functionality:
  - `pico_btstack_ble` - Adds Bluetooth Low Energy (LE) support.
  - `pico_btstack_classic` - Adds Bluetooth Classic support.
  - `pico_btstack_sbc_encoder` - Adds Bluetooth Sub Band Coding (SBC) encoder support.
  - `pico_btstack_sbc_decoder` - Adds Bluetooth Sub Band Coding (SBC) decoder support.
  - `pico_btstack_bnep_lwip` - Adds Bluetooth Network Encapsulation Protocol (BNEP) support using LwIP.
  - `pico_btstack_bnep_lwip_sys_freertos` - Adds Bluetooth Network Encapsulation Protocol (BNEP) support using LwIP with FreeRTOS for `NO_SYS=0`.
- The following integration libraries are also provided:
  - `pico_btstack_run_loop_async_context` - provides a common `async_context` backed implementation of a BTstack *"run loop"* that can be used for all BTstack use with the `pico-sdk`.
  - `pico_btstack_flash_bank` - provides a sample implementation for storing required Bluetooth state in flash.
  - `pico_btstack_cyw43` - integrates BTstack with the CYW43 driver.
- The `CMake` function `pico_btstack_make_gatt_header` can be used to run the BTstack `compile_gatt` tool to make a GATT header file from a BTstack `GATT` file.

- `pico_cyw43_driver` and `cyw43_driver` now support HCI communication for Bluetooth.
- `cyw43_driver_picow` now supports Pico W specific HCI communication for Bluetooth over SPI.
- `cyw43_arch_init()` and `cyw43_arch_deinit()` automatically handle Bluetooth support if `CYW43_ENABLE_BLUETOOTH` is `1` (as it will be automatically if you depend on `pico_btstack_cyw43`).

Key changes since 1.5.0:

- Added Raspberry Pi specific BTstack license.
- The storage offset in flash for `pico_btstack_flash_bank` can be specified at runtime by defining `pico_flash_bank_get_storage_offset_func` to your own function to return the offset within flash.
- `pico_btstack_flash_bank` is now safe for multicore / FreeRTOS SMP use, as it uses the new `pico_flash` library to make sure the other core is not accessing flash during flash updates. If you are using `pico_multicore` you must have called `flash_safe_execute_core_init` from the "other" core (to the one Bluetooth is running on).
- Automatically set Bluetooth MAC address to the correct MAC address (Wi-Fi MAC address + 1), as some devices do not have it set in OTP and were using the same default MAC from the Bluetooth chip causing collisions.
- Various bug-fixes and stability improvements (especially with concurrent Wi-Fi), including updating `cyw43_driver` and `btstack` to the newest versions.

Release 2.0.0 (08 August 2024)

This is a major release which adds support for the new RP2350 and for compiling RISC-V code in addition to Arm.

- There is a lot of new functionality in the RP2350 microcontroller, it is recommended that you read the RP2350 Datasheet
- There is a lot of new functionality in the SDK, it is also worth reading the Raspberry Pi Pico-series C/C++ SDK book. This also includes documentation for RP2040 and RP2350 APIs, along with much more complete documentation of SDK `#defines` and `CMake` build variables.

Notices

IMPORTANT

You should delete/recreate all build directories when upgrading from previous versions of the Raspberry Pi Pico SDK.

Major New Features

Support for RP2350

Many programs you have written for RP2040 (say a Raspberry Pi Pico) should work unmodified on RP2350 (say a Raspberry Pi Pico 2) even when compiled for RISC-V.

- You can now specify `rp2350-arm-s` (Arm Secure) or `rp2350-riscv` (RISC-V) as well as the previous `rp2040` (default) and `host`.
- Setting `PICO_BOARD=some_board` will now set `PICO_PLATFORM` if one is specified in `some_board.h` since most boards either use exclusively RP2040 or RP2350.
- `PICO_PLATFORM` also supports `rp2350` but this gets replaced with the value `PICO_DEFAULT_RP2350_PLATFORM` which you can set in your environment or `CMakeLists.txt`. Many of the boards for RP2350 - including `pico2`- select `rp2350` as the `PICO_BOARD` to honour your preference.

NOTE

This release of the SDK does not support writing Arm Non-Secure binaries to run under the wing of an Arm Secure binary. This support will be added in a subsequent release.

Security and Code Signing

- The RP2350 bootrom contains support for signed images and a variety of other security features. The SDK supports building signed images etc. as part of the CMake build. For further information, please read RP2350 Datasheet "Bootrom Concepts" section, and also the Raspberry Pi Pico-series C/C++ SDK book for details on configuring your build to sign code. Note that signed code is only applicable to chips that have been locked down for security, but you can also hash your image for integrity checking.

Board Support

The following boards have been added and may be specified via `PICO_BOARD`:

- `defcon32_badge`
- `gen4_rp2350_24`
- `gen4_rp2350_24ct`
- `gen4_rp2350_24t`
- `gen4_rp2350_28`
- `gen4_rp2350_28ct`
- `gen4_rp2350_28t`
- `gen4_rp2350_32`
- `gen4_rp2350_32ct`
- `gen4_rp2350_32t`
- `gen4_rp2350_35`
- `gen4_rp2350_35ct`
- `gen4_rp2350_35t`
- `hellbender_2350A_devboard`
- `ilabs_challenger_rp2350_bconnect`
- `ilabs_challenger_rp2350_wifi_ble`
- `melopero_perpetuo_rp2350_lora`
- `phyx_rick_tny_rp2350`
- `pico2`
- `pimoroni_pga2350`
- `pimoroni_pico_plus2_rp2350`
- `pimoroni_plasma2350`
- `pimoroni_tiny2350`
- `seeed_xiao_rp2350`
- `solderparty_rp2350_stamp`

- solderparty_rp2350_stamp_xl
- sparkfun_promicro_rp2350
- switchscience_picossci2_conta_base
- switchscience_picossci2_dev_board
- switchscience_picossci2_micro
- switchscience_picossci2_rp2350_breakout
- switchscience_picossci2_tiny
- tinycircuits_thumby_color_rp2350

New Libraries

hardware_boot_lock (RP2350)

- New library for accessing the BOOT locks from secure code.

hardware_dcp (RP2350 Arm)

- Contains assembler macros for individual DCP (Double Co-Processor) instructions
- Contains assembler macros for canned instruction sequences for higher-level operations
- HAS_DOUBLE_COPROCESSOR define indicates hardware support

hardware_hazard3 (RP2350 RISC-V)

- Assembler macros and inline functions for accessing Hazard3 extensions

hardware_powman (RP2350)

- Hardware APIs for the Power Management hardware.
- HAS_POWMAN_TIMER define indicates hardware support.

hardware_rcp (RP2350 Arm)

- Contains inline functions and assembler macros for the RCP (Redundancy Co-Processor) instructions.
- HAS_REDUNDANCY_COPROCESSOR define indicates hardware support.

hardware_riscv_platform_timer (RP2350)

- Hardware APIs for the RISC-V Platform Timer (which is also made available on Arm).

hardware_sha256 (RP2350)

- Hardware APIs for the SHA256 hashing hardware.

hardware_ticks

- Hardware APIs for the RP2350 tick generators.
- On RP2040 the same API is used, but only one tick generator TICK_WATCHDOG is used, which is backed by the hardware in the RP2040 WatchDog hardware.

pico_aon_timer

- Abstraction for a hardware timer that is "Always-On", and can wake the processor up even from a low power state at a given time.
  - On RP2040 this uses the RTC.
  - On RP2350 this uses the Powman Timer.

pico_atomic

- Additional support for C11 atomic functions using spin lock number PICO_SPINLOCK_ID_ATOMIC.
  - On RP2040, all functions are implemented via spinlock.
  - On RP2350, only 64-bit or arbitrary-sized atomics are implemented via spin lock; the rest use processor exclusive/atomic instructions.
    - ACTLR.EXTEXCLALL must be set to 1 on each processor for the exclusive instructions to work. This is done automatically in the SDK by one of the per-core initializers in pico_runtime_init.
- Included by pico_runtime by default.

pico_boot_lock (RP2350)

- Support for acquiring and releasing locks to prevent concurrent use of hardware resources used by bootrom functions.
- Enabled via PICO_BOOTROM_LOCKING_ENABLED which defaults to 1 on RP2350.
- Some bootrom functions use shared resources such as the single SHA256 or put hardware such as the OTP or XIP interface into a state that cannot execute concurrently with certain other code. The bootrom supports checking that the resource is owned, and this library turns that checking on.
- The bootrom function wrappers in pico_bootrom call the functions in pico_boot_lock around affected bootrom functions, and thus will take and release locks if PICO_BOOTROM_LOCKING_ENABLED=1.
- NUM_BOOT_LOCKS define indicates the number of boot locks (8 on 'RP2350', 0 on 'RP2040').

pico_clib_interface

- New library to encapsulate the interface between the SDK and the C library.
- Supports
  - newlib (full).
  - picolibc (preview).
  - llvm-libc (preview).
- Included by pico_runtime by default.

pico_crt0

- New library split out of `pico_standard_link` to encapsulate the earliest startup code before the runtime initialisation, and shutdown code after the runtime.
- Repository for the default RP2040 and RP2350 linker scripts.
  - The flash size specified in the board header is now used when linking which is handy if you have >2MB of flash and >2MB of code/data.
  - **Note:** The linker scripts have changed since the previous release of the SDK. If you have custom linker scripts, it is recommended that you update them to match.
    - In particular the new linker scripts include an "embedded block" which is required for a binary to boot on RP2350.
    - `__HeapLimit` is now defined to be the end of RAM rather than the end of a `PICO_HEAP_SIZE` chunk, to better match the standard behaviour. `PICO_HEAP_SIZE` is the minimum heap size required, and space is required for it at link time. `sbrk` in the previous SDK ignored it anyway and used the end of RAM so there is no functional change there.
- Included by `pico_runtime` by default

pico_cxx_options

- New library split out of `pico_standard_link` to configure C++ options.
- Included by `pico_standard_link` by default.

pico_platform_compiler

- New library split out of `pico_platform` with the functions/macros related to the compiler.
- Included by `pico_platform` by default.

pico_platform_panic

- New library split out of `pico_platform` with the panic function implementation.
- Included by `pico_platform` by default.

pico_platform_sections

- New library split out of `pico_platform` with the section macros such as `__not_in_flash_func`.
- Included by `pico_platform` by default.

pico_runtime_init

- Contains the standard initialisers that should get run before main, or per core.
- Unlike in the previous SDK version where `runtime_init()` was a monolithic function which also called some `__preinit_array` initialisers, the new runtime_init library:
  - Separates each initialiser out individually, for say initialiser "foo".
    - Defines `PICO_RUNTIME_INIT_FOO` which is a "12345" *line number* ordering of the initialiser with respect to others.
    - Declares `runtime_init_foo()` which is the actual initialiser.

    - If PICO_RUNTIME_SKIP_INIT_FOO is not set, it adds the initialiser entry to call runtime_init_foo() before main (or per core initialisation).
    - If PICO_RUNTIME_NO_INIT_FOO is not set, it adds the (weak) implementation of runtime_init_foo().
    - This gives the user full control to customise runtime initialisation, either skipping or replacing parts.
- Included by pico_runtime by default.

pico_sha256

- High level APIs for generating SHA256 hashes both synchronously and asynchronously

pico_standard_binary_info

- New library split out of pico_standard_link that adds the "common" binary info items to the binary.
- Included by pico_standard_link by default.

Library Changes / Improvements

Note that all hardware libraries now support the increased number of GPIOs on RP2350B in APIs that take a GPIO number; this is not noted for every library.

pico_base

- More error return codes were added to pico/error.h, mostly because these are the same values returned by RP2350 bootrom API functions, but also a number of new SDK APIs also return meaningful errors.
- In pico/types.h, by popular demand, absolute_time_t now always defaults to uint64_t regardless of the type of build. You can set PICO_OPAQUE_ABSOLUTE_TIME_T=1 to make it a struct in all build types.

pico_binary_info

- Now supports > 32 GPIO pins when PICO_BINARY_INFO_USE_PINS_64=1 - this is defaulted for you based on the number of GPIOs on the board.

hardware_adc

- PARAM_ASSERTIONS_ENABLED_ADC is renamed to PARAM_ASSERTIONS_ENABLED_HARDWARE_ADC - the old define is still supported as a fallback.
- ADC_TEMPERATURE_CHANNEL_NUM added since this value varies between RP2040 and RP2350.

hardware_clocks

- set_sys_clock_ functions are now in hardware/clocks.h.
- Clock configuration.
    - PLL_COMMON_REFDIV is deprecated in favour of PLL_SYS_REFDIV and PLL_USB_REFDIV.
    - PLL_SYS_VCO_FREQ_HZ is new and preferred over PLL_SYS_VCO_FREQ_KHZ.
    - PLL_USB_VCO_FREQ_HZ is new and preferred over PLL_USB_VCO_FREQ_KHZ.
    - XOSC_HZ, SYS_CLK_HZ, USB_CLK_HZ now added, and take preference over the still supported XOSC_KHZ, SYS_CLK_KHZ, and

USB_CLK_KHZ.

- set_sys_clock_hz() and check_sys_clock_hz() added.
- clock_configure_undivided() and clock_configure_int_divider() for no divisor or a whole integer divider as the code doesn't require 64-bit arithmetic and thus saves space.
- The enum clock_index no longer exists and has been replaced with clock_num_t. However, all clock functions now take clock_handle_t to allow for future enhancement. This is currently just an alias for clock_num_t
- vcocalc.py can now be used to generate the CMake configuration for a particular clock setting.
- The default system clock on RP2350 is 150 MHz.

hardware_divider

- Since the RP2350 processors have efficient divider instructions, RP2350 has no SIO HW Divider. Software versions of the hardware_divider functions are provided for RP2350.
- HAS_SIO_DIVIDER define is now provided for you.

hardware_dma

- PARAM_ASSERTIONS_ENABLED_DMA is renamed to PARAM_ASSERTIONS_ENABLED_HARDWARE_DMA - the old define is still supported as a fallback.
- Added dma_get_irq_num()`function and `DMA_IRQ_NUM() macro to return the process IRQ Number for the *n* th DMA IRQ.
- NUM_DMA_IRQS define is provided for you.
  - it is 2 on RP2040 and 4 on RP2350.

hardware_exception

- PARAM_ASSERTIONS_ENABLED_EXCEPTION is renamed to PARAM_ASSERTIONS_ENABLED_HARDWARE_EXCEPTION - the old define is still supported as a fallback.
- Added RISC-V support.
  - exception numbers are processor exception cause numbers.
- exeception_[get|set]_priority() are added for Arm.

hardware_flash

- PARAM_ASSERTIONS_ENABLED_FLASH is renamed to PARAM_ASSERTIONS_ENABLED_HARDWARE_FLASH - the old define is still supported as a fallback.
- flash_flush_cache() is added.

hardware_gpio

- PARAM_ASSERTIONS_ENABLED_GPIO is renamed to PARAM_ASSERTIONS_ENABLED_HARDWARE_GPIO - the old define is still supported as a fallback.
- The enum gpio_function no longer exists and has been replaced with gpio_function_t.
- gpio_xxx_masked() functions now have a gpio_xxx_masked64() variant that takes a 64-bit mask of GPIO indexes.
- gpio_xxx_mask() functions now have a gpio_xxx_mask64() variant that takes a 64-bit mask of GPIO indexes.
- gpio_get_all64() added to read the state of >32 pins.

- gpio_put_all64() added to write the state of >32 pins.
- On Arm RP2350 GPIO Co-Processor instructions are used by default. This is controlled via PICO_USE_GPIO_COPROCESSOR.
- HAS_GPIO_COPROCESSOR define indicates hardware support.

hardware_i2c

- PARAM_ASSERTIONS_ENABLED_I2C is renamed to PARAM_ASSERTIONS_ENABLED_HARDWARE_I2C - the old define is still supported as a fallback.
- PICO_DEFAULT_I2C_INSTANCE() macro added which is equivalent to the pre-existing i2c_default
- Added I2C_NUM(), I2C_INSTANCE(), I2C_DREQ_NUM() macros to abstract differences between platforms.
- Fixed per-character timeouts.

hardware_interp

- PARAM_ASSERTIONS_ENABLED_INTERP is renamed to PARAM_ASSERTIONS_ENABLED_HARDWARE_INTERP - the old define is still supported as a fallback.

hardware_irq

- PARAM_ASSERTIONS_ENABLED_IRQ is renamed to PARAM_ASSERTIONS_ENABLED_HARDWARE_IRQ - the old define is still supported as a fallback.
- irq_xxx_mask_xxx() functions now have a gpio_xxx_mask_n_xxx() variant that affects the *n* th set of 32 IRQs
- Expose runtime_init_per_core_irq_priorities() function
- Added irq_set_riscv_vector_handler() function to replace code entries in the machine vector table.

hardware_pio

- PARAM_ASSERTIONS_ENABLED_PIO is renamed to PARAM_ASSERTIONS_ENABLED_HARDWARE_PIO - the old define is still supported as a fallback.
- PICO_PIO_VERSION is used to determine whether new RP2350 functionality (PICO_PIO_VERSION=1) is supported. This is defaulted based on the platform.
- PICO_PIO_USE_GPIO_BASE is used to determine whether support is enabled for GPIOs above 32. TThe default value is set based on the chip package.
- Added pio_sm_set_jmp_pin().
- Added pio_claim_free_sm_and_add_program(), pio_claim_free_sm_and_add_program_for_gpio_range() and `pio_remove_program_and_unclaim_sm() to simplify finding and claiming a free PIO instance and state machine and installing programs.
- Added `pio_get_irq_num()`function to return the process IRQ Number for the *n* th PIO IRQ for a PIO instance.
- Added PIO_NUM(), PIO_INSTANCE(), PIO_IRQ_NUM(), PIO_DREQ_NUM() and PIO_FUNCSEL_NUM() macros to abstract differences between platforms.
- Added sm_config_set_out_pin_base() and sm_config_set_out_pin_count().
- Added sm_config_set_in_pin_base() and sm_config_set_in_pin_count(). Note the latter is only meaningful on PICO_PIO_VERSION=1 which supports a limit.
- Added sm_config_set_set_pin_base() and sm_config_set_set_pin_count().

- Added `sm_config_set_sideset_pin_base()` and `sm_config_set_sideset_pin_count()`.
- For `PICO_PICO_VERSION=1` i.e. RP2350:
  - Added `pio_set_gpio_base()` and `pio_get_gpio_base()` to assign the PIO instance to pins 0-31 or 16-47.
  - Added `pio_set_sm_multi_mask_enabled()`.
  - Added `pio_clkdiv_restart_sm_multi_mask()`.
  - Added `pio_enable_sm_multi_mask_in_sync()`.
- `NUM_PIO_IRQS` define is now provided for you (2 on both RP2040 and RP2350).

hardware_pll

- `PICO_PLL_VCO_MIN_FREQ_HZ` is new and now preferred to `PICO_PLL_VCO_MIN_FREQ_KHZ` or `PICO_PLL_VCO_MIN_FREQ_MHZ`.
- `PICO_PLL_VCO_MAX_FREQ_HZ` is new and now preferred to `PICO_PLL_VCO_MAX_FREQ_KHZ` or `PICO_PLL_VCO_MAX_FREQ_MHZ`.
- `PLL_RESET_NUM()` macro added to abstract differences between platforms.

hardware_pwm

- `PARAM_ASSERTIONS_ENABLED_PWM` is renamed to `PARAM_ASSERTIONS_ENABLED_HARDWARE_PWM` - the old define is still supported as a fallback.
- `PICO_DEFAULT_PWM_INSTANCE()` macro added which is equivalent to the pre-existing `pwm_default`.
- Added `PWM_SLICE_NUM()` and `PWM_DREQ_NUM()` macros to abstract differences between platforms.
- Added `PWM_DEFAULT_IRQ_NUM()` since RP2350 supports 2 PWM IRQs to indicate which IRQ the pre-existing RP2040 functions use.
- Added `pwm_set_irq0_enabled()`, `pwm_set_irq1_enabled()` and `pwm_irqn_set_slice_enabled()` to differentiate between the IRQs.
- Added `pwm_set_irq0_mask_enabled()`, `pwm_set_irq1_mask_enabled()` and `pwm_irqn_set_mask_enabled()` to differentiate between the IRQs.
- Added `pwm_get_irq0_status_mask()`, `pwm_get_irq1_status_mask()` and `pwm_irqn_get_status_mask()` to differentiate between the IRQs.
- Added `pwm_pwm_force_irq0()`, `pwm_force_irq1()` and `pwm_irqn_force()` to differentiate between the IRQs.

hardware_resets

- `PARAM_ASSERTIONS_ENABLED_RESETS` is renamed to `PARAM_ASSERTIONS_ENABLED_HARDWARE_RESETS` - the old define is still supported as a fallback.
- `reset_block()` is renamed to `reset_block_mask()` but the old name is still supported.
- `unreset_block()` is renamed to `unreset_block_mask()` but the old name is still supported.
- `unreset_block_wait()` is renamed to `unreset_block_mask_wait_blocking()` but the old name is still supported.
- `reset_block_num()`, `unreset_block_num()`, `unreset_block_num_wait_blocking()` and `reset_unreset_block_num_wait_blocking()` added to reset or unreset a single block by `reset_num_t` index.

hardware_rtc

- Note this library is only available on RP2040, since the RP2350 lacks the RTC hardware.
- There is a similar always-on timer in `hardware_powman`.

- A common API for both RP2040 and RP2350 is provided in `pico_aon_timer`.
- `HAS_RP2040_RTC` define is now provided for you.

hardware_spi

- `PARAM_ASSERTIONS_ENABLED_SPI` is renamed to `PARAM_ASSERTIONS_ENABLED_HARDWARE_SPI` - the old define is still supported as a fallback.
- `PICO_DEFAULT_SPI_INSTANCE()` macro added which is equivalent to the pre-existing `spi_default`.
- Added `SPI_NUM()`, `SPI_INSTANCE()`, `SPI_DREQ_NUM()` macros to abstract differences between platforms.
- Fixed per-character timeouts.

hardware_sync

- `restore_interrupts_from_disabled()` is added as a variant for `restore_interrupts()` which **must** be paired with a matching `save_and_disable_interrupts()`. This is the common usage and produces smaller/faster code on RISC-V.
- Spinlock functionality has been delegated to a separate hardware_sync_spinlock library, which is included for you.
- hardware_sync_spin_lock.
  - Whilst RP2350 has the same SIO spin locks as RP2040, due to Errata RP2350-E2, these are not used by default.
  - Instead, a software implementation using atomic instructions is used.
  - You can set `PICO_USE_SW_SPIN_LOCKS=0` to disable this if you know you aren't affected by RP2350-E2 and want to use the h/w spin locks instead.
  - Added `spin_try_lock_unsafe()` function.

hardware_timer

- `PARAM_ASSERTIONS_ENABLED_TIMER` is renamed to `PARAM_ASSERTIONS_ENABLED_HARDWARE_TIMER` - the old define is still supported as a fallback.
- RP2350 supports two timer instances.
  - `PICO_DEFAULT_TIMER_INSTANCE()` macro added based on `PICO_DEFAULT_TIMER` (0 on RP2040, 0/1 on RP2350).
  - Added `TIMER_NUM()`, `TIMER_INSTANCE()`, `TIMER_ALARM_NUM_FROM_IRQ()` and `TIMER_ALARM_NUM_FROM_IRQ()` macros to abstract differences between platforms
  - Added `hardware_alarm_get_irq_num()` to get the processor IRQ number for a particular alarm on a timer.
  - New versions of all functions added with a `timer_` prefix and a timer instance passed as the first argument. The pre-existing functions call these with the default timer instance.
- `NUM_TIMERS` has been renamed to `NUM_ALARMS` as that's what it was (4).
- `NUM_GENERIC_TIMERS` has been added which is 1 on RP2040 and 2 on RP2350.

hardware_uart

- `PARAM_ASSERTIONS_ENABLED_UART` is renamed to `PARAM_ASSERTIONS_ENABLED_HARDWARE_UART` - the old define is still supported as a fallback.
- `PICO_DEFAULT_UART_INSTANCE()` macro added which is equivalent to the pre-existing `uart_default`.
- Added `UART_NUM()`, `UART_INSTANCE()`, `UART_DREQ_NUM()`, `UART_IRQ_NUM()`, `UART_CLOCK_NUM()`, `UART_RESET_NUM()`, `UART_FUNCSEL_NUM()` macros to abstract differences between platforms.

- `uart_set_irq_enables()` is renamed to `uart_set_irqs_enabled()` but the old name is still supported.
- `uart_get_dreq()` is renamed to `uart_get_dreq_num()` but the old name is still supported.
- `uart_get_reset_num()` is added.
- Incorrect baud setting for certain frequencies was fixed.

hardware_vreg

- `vreg_disable_voltage_limit()` added to allow full range of DVDD voltage selection on RP2350

hardware_watchdog

- `PARAM_ASSERTIONS_ENABLED_WATCHDOG` is renamed to `PARAM_ASSERTIONS_ENABLED_HARDWARE_WATCHDOG` - the old define is still supported as a fallback.
- Added `watchdog_disable()`.
- `watchdog_get_count()` is renamed to `watchdog_get_time_remaining_ms()` but the old name is still supported.

hardware_xosc

- `XOSC_HZ` is new and now preferred to `XOSC_KHZ`.

hardware_regs

- `enum irq_num_[rp2040|rp2350]` (typedef-ed as `irq_num_t`) added with the constants from `inctrl.h`. Note these remain as `#defines` when included from assembly.
- `enum dreq_num_[rp2040|rp2350]` (typedef-ed as `dreq_num_t`) added with the constants from `dreq.h`. Note these remain as `#defines` when included from assembly.

hardware_structs

- `enum bus_ctrl_perf_counter_[rp2040|rp2350]` (typedef-ed as `bus_ctrl_perf_counter_t`) added.
  - **Note** enum bus_ctrl_per_counter no longer exists.
- `enum clock_num_[rp2040|rp2350]` (typedef-ed as `clock_num_t`) added.
  - **Note** enum clock_index no longer exists.
- `enum clock_dest_num_[rp2040|rp2350]` (typedef-ed as `clock_dest_num_t`) added.
- `enum gpio_function_[rp2040|rp2350]` (typedef-ed as `gpio_function_t`) added.
  - **Note** enum gpio_function no longer exists.
- `enum gpio_function1_[rp2040|rp2350]` (typedef-ed as `gpio_function1_t`) added (for QSPI bank).
- `enum reset_num_[rp2040|rp2350]` (typedef-ed as `reset_num_t`) added.
- `enum tick_gen_num_rp2350` (typedef-ed as `reset_num_t`) added.
- Various naming consistencies have been fixed.
  - `iobank0.h` → `io_bank00.h`, `iobank0_hw` → `io_bank0_hw` - shims are provided for the old versions.
  - `ioqspi.h` → `io_qspi0.h`, `ioqspi_hw` → `io_qspi_hw` - shims are provided for the old versions.
  - `padsbank0.h` → `pads_bank0.h`, `padsbank0_hw` → `pads_bank0_hw` - shims are provided for the old versions.

- padsqspi.h → pads_qspi.h, padsqspi_hw → pads_qspi_hw - shims are provided for the old versions.
- bus_ctrl.h → busctrl.h, bus_ctrl_hw → busctrl_hw (don't ask! but hardware_struct headers now match hardware_regs names at least!).

boot_stage2

- There are now separate implementations for RP2040 and RP2350.
- A boot_stage2 is not needed on RP2350, but one can be included via the define PICO_EMBED_XIP_SETUP=1.

cmsis

- CMSIS headers are updated to CMSIS 6.1
- Device headers RP2040.h and RP2350.h are generated, and now include basic hardware structures as per the latest SVDConv defaults.

pico_bootrom

- New RP2350 bootrom APIs added.
- rom_xxx() inline function wrappers added for all xxx() ROM functions.
- Additional rom_get_boot_random() and rom_add_flash_runtime_partition() for RP2350 which use underlying bootrom functionality but aren't just wrapper functions.

pico_bt_stack

- BTStack updated to 1.6.1 from 1.5.6
  - Lots of additions, fixes and changes, for the full list see the change log

pico_cyw43_arch

- PARAM_ASSERTIONS_ENABLED_CYW43_ARCH is renamed to PARAM_ASSERTIONS_ENABLED_PICO_CYW43_ARCH - the old define is still supported as a fallback.
- lib/cyw43-driver has been updated to the latest version
  - Mostly bug fixes.
  - Adds WPA3 support for Pico W. To use this, use CYW43_AUTH_WPA3_SAE_AES_PSK or CYW43_AUTH_WPA3_WPA2_AES_PSK instead of CYW43_AUTH_WPA2_AES_PSK when connecting to wifi with cyw43_arch_wifi_connect_timeout_ms or cyw43_arch_enable_ap_mode.

pico_cyw43_driver

- cyw43_driver updated to commit faf36381.
- Added support for changing the clock speed of the SPI connection to the Wi-Fi chip. See CYW43_PIO_CLOCK_DIV_INT, CYW43_PIO_CLOCK_DIV_FRAC and CYW43_PIO_CLOCK_DIV_DYNAMIC.

pico_divider

- Functions that returned a quotient and divider in a uint64_t or int64_t now return a divmod_result_t - the signed-ness of the value before was meaningless anyway, and the compiler will still return it as a 64-bit value.

- Extra functions in `pico/divider.h` now implemented for `pico_set_divider_implemtation(compiler)` as well as for RP2350 which has no RP2040 hardware divider.

pico_double

- `pico_set_double_implementation(pico)` (the default) now uses the Double Co-Processor (DCP) for double-precision floating-point arithmetic on Arm RP2350, and highly optimised Arm VFP implementations of the double-precision scientific functions, for much improved performance over the C library versions.
- Extra functions exposed from `pico` implementation
  - `int2double()`
  - `uint2double()`
  - `int642double()`
  - `uint642double()`
  - `double2uint()`
  - `double2uint64()`
- Extra functions exposed from `pico` implementation for Arm RP2350 only
  - `ddiv_fast()`
  - `sqrt_fast()`
  - `mla()`

pico_float

- `pico_set_float_implementation(pico)` (the default) now uses the compiler for single-precision floating point arithmetic on Arm RP2350 since the processor has VFP instructions, but includes custom optimised scientific functions also using the VFP.
- `pico_set_diouble_implementation(pico_dcp)` uses the Double Co-Processor (DCP) for single-precision floating point arithmetic on Arm RP2350, and highly optimised Arm M33 implementations of the single-precision scientific functions, for much improved performance over the C library versions. This library is intended for those situations where you cannot (or don't want to) use the VFP instructions.
- Extra functions exposed from `pico` implementation.
  - `int2float()`
  - `uint2float()`
  - `int642float()`
  - `uint642float()`
  - `float2uint()`
  - `float2uint64()`
  - `float2uint_z()`
  - `float2uint64_z()`
- Extra functions exposed from `pico` implementation for Arm R2350 only.
  - `float2fix64_z()`
  - `fdiv_fast()`
  - `fsqrt_fast()`

pico_lwip

- Update lib/lwip to 2.2.0
  - There have been some bugs fixed, and some new features were added (most notably full Address Conflict Detection support).

pico_mbedtls

- Update to lib/mbedtls to 2.28.8 from 2.28.1
  - This release of Mbed TLS provides bug fixes and minor enhancements. This release includes fixes for security issues.
- Added support for hardware SHA256 calculation on RP2350
  - To use this in mbedtls you need to define MBEDTLS_SHA256_ALT in your mbedtls_config.h. Use LIB_PICO_SHA256 to check if hardware SHA256 is supported and fallback to defining MBEDTLS_SHA256_C for the software SHA256 calculation.

pico_multicore

- Added SIO_FIFO_IRQ_NUM() to get the IRQ number for the FIFO IRQ on a particular core, since RP2040 and RP2350 are different.
  - **note** that RP2350 uses the same IRQ number on both cores, so if you have IRQ handlers for both cores, you should share the same function and check the core number in the IRQ handler. This strategy of course works on RP2040 too.
- Added multicore_fifo_push_blocking_inline() and multicore_fifo_pop_blocking_inline().
- Added multicore_doorbell_ functions for the new inter-core Doorbells on RP2350.
  - NUM_DOORBELLS is provided which is 8 on RP2350, 0 on RP2040.

pico_rand

- Added the hardware TRNG as an additional entropy source on RP2350.
  - HAS_RP2350_TRNG indicates hardware support.
- Many, but not all, of the pre-existing entropy sources are disabled on RP2350 in favour of using the TRNG.

pico_runtime

- A shadow of its former self, it now just:
  - aggregates other default libraries required for getting to main() and having the C runtime work.
  - provides low level runtime_run_initializers() and runtime_run_per_core_initializers() which run initializers from the __preinit_array.
- The runtime_init() entrypoint has moved to pico_clib_interface.

pico_standard_link

- Much previously included functionality has been split out into pico_crt0, pico_cxx_options and pico_standard_binary_info.
- What remains is entirely focused on setting up the linker configuration.

- **Finally** fixed a bug where changes to the linker script did not cause a relink.

pico_stdio

- Some internal reorganisation to separate functionality between here and pico_clib_interface.
- Added PICO_STDIO_SHORT_CIRCUIT_CLIB_FUNCS to control whether printf, vprintf, puts, putchar and getchar go thru the C library (thus usually pulling in all the FILE handling APIs resulting in huge bloat - but more sensible behaviour when mixing say printf with fprintf(stdout etc.) This defaults to 0, i.e. "do short-circuit the c lib" which was the behaviour in the previous SDK version.
- Add support for Segger RTT stdio.
- Implemented stdio_flush() for UART and USB CDC.
- Added stdio_deinit_all() and individual stdio_deinit_xxx functions.

pico_stdio_usb

- Now supports MS OS2 descriptors by default. See PICO_STDIO_USB_RESET_INTERFACE_SUPPORT_MS_OS_20_DESCRIPTOR.
- PICO_STDIO_USB_ENABLE_RESET_VIA_VENDOR_INTERFACE and PICO_STDIO_USB_ENABLE_RESET_VIA_BAUD_RATE are now both supported even if the user is using tinyusb_device directly themselves.
- Bug that could cause deadlock with FreeRTOS SMP and printing from IRQs fixed.

pico_stdlib

- pico/stdlib.h no longer declares set_sys_clock_ functions. You must include hardware/clocks.h explicitly.

pico_time

- remaining_alarm_time_ms(), remaining_alarm_time_us(), alarm_pool_remaining_alarm_time_ms() and alarm_pool_remaining_alarm_time_us() were added.
- Implementation of alarm pools completely rewritten for much lower overhead, jitter and higher throughput in the majority of cases. The pairing heap has been replaced with a linked list which is faster and uses less memory in most normal use cases too.

IMPORTANT

fire_if_past now always fires asynchronously in the same way as a normal timeout (rather than being called synchronously during the call). Thus alarm_pool_add_alarm_at_force_in_context is now no different to alarm_pool_add_alarm_at.

- New pico_timer_adapter abstraction added so pico_time could be backed by other types of timer hardware in the future, and so pico_time no longer depends directly on a hardware_timer abstraction which simplifies PICO_PLATFORM=host.
- Support for two hardware timer blocks on RP2350.
  - alarm_pool_timer_t abstraction added to represent the time "counter" backing the alarm pool.
  - alarm_pool_t now has an associated alarm_pool_timer_t instance.
  - alarm_pool_create_on_timer() is added to create an alarm pool on a specific alarm pool timer.
  - alarm_pool_get_default_timer() is added which is used when not explicitly passing an alarm pool timer. PICO_DEFAULT_TIMER selects which timer instance is the default (0 on RP2040, 0/1 on RP2350).

- PARAM_ASSERTIONS_ENABLED_TIME is renamed to PARAM_ASSERTIONS_ENABLED_PICO_TIME - the old define is still supported as a fallback.
- check_timeout_fn now takes two parameters. This was likely unused outside the pico_time implementation anyway.
- Expose runtime_init_default_alarm_pool() function.

pico_util

- time_to_datetime(), datetime_to_time() and datetime_to_str() functions relating to hardware_rtc are now guarded by PICO_INCLUDE_RTC_DATETIME which defaults to 0 on RP2350, since RP2350 does not include the RP2040 RTC hardware.
- timespec_to_ms(), timespec_to_us(), ms_to_timespec(), and ms_to_timespec() added to convert between C-library high-resolution time offset and millisecond or microsecond precision offsets.
- queue_try_remove(), queue_remove_blocking() and queue_peek_blocking() now support passing NULL as the element out pointer if the caller doesn't care.

tinyusb

- TinyUSB moved from release 0.15.0 to commit 42326428 (0.17.0 WIP)
- Note that bsp/board.h has been renamed by TinyUSB to bsp/board_api.h the SDK adds a re-director header for you for now.
- Support added for RP2350. Requires a custom memcpy implementation in the rp2040 tinusb driver, as unaligned 32 bit access to device memory causes a hard fault on the Cortex M33.
- See the TinyUSB changelog for full details.

pioasm

- pioasm now supports the full RP2350 PIO (PICO_PIO_VERSION=1) instruction set
- Additionally, it supports many new directives. See the RP2350 Datasheet for full details.

NOTE

currently not all output formats support PICO_PIO_VERSION=1 as they are community provided.

FreeRTOS integration

- You should use this repo for the current FreeRTOS-Kernel supporting RP2040 and RP2350: https://github.com/raspberrypi/FreeRTOS-Kernel.
- Dropped legacy support for configNUM_CORES for the correct configNUMBER_OF_CORES, which is 2 for SMP support and 1 for non-SMP support.
- RP2350_ARM_NTZ (non-trust-zone), and RP2350_RISC-V are available as well as an updated RP2040 version; the former two basically give you the same "single privilege/security domain" experience as on RP2040.
- SMP and non-SMP support (along with running FreeRTOS on either core) are available for all.
- A nasty, but rare pre-existing RP2040 deadlock (especially with TinyUSB printf from IRQs) has been fixed on all three versions; If you were setting configSUPPORT_PICO_SYNC_INTEROP=0 as a workaround, you should no longer do so. Generally, if you are using printf (or anything else using SDK locking primitives) then you do really want configSUPPORT_PICO_SYNC_INTEROP=1 for the best concurrency
- FreeRTOS on RISC-V does not currently support IRQ preemption (which is a Hazard3 only feature anyway).

Backwards Incompatibilities

There are a handful of minor backwards incompatibilities, that hopefully should affect very few people:

- `boot_picobin` library is now called `boot_picobin_headers`.
- `boot picoboot` library is now called `boot_picoboot_headers`.
- `boot_uf2` library is now called `boot_uf2_headers`.
- `pico_base` library is now called `pico_base_headers`.
  - `pico/error.h` - `PICO_ERROR_GENERIC` is now `-1` because there were pre-existing APIs that returned -1 for any error. `PICO_ERROR_TIMEOUT` is now `-2` (they are swapped from their previous values).
- `pico_stdlib`
  - `pico/stdlib.h` no longer declares `set_sys_clock_` functions. You must include `hardware/clocks.h` explicitly.
- `pico_time`
  - `check_timeout_fn` now takes two parameters. This was likely unused outside the `pico_time` implementation anyway.
  - `fire_if_past` now always fires asynchronously in the same way as a normal timeout (rather than being called synchronously during the call). Thus `alarm_pool_add_alarm_at_force_in_context` is now no different to `alarm_pool_add_alarm_at`.
- `hardware_clocks`
  - The `enum clock_index` no longer exists and has been replaced with `clock_num_t`. However, all clock functions now take `clock_handle_t` to allow for future enhancement. This is currently just an alias for `clock_num_t`.
- `hardware_structs`
  - `enum bus_ctrl_perf_counter_[rp2040|rp2350]` (typedef-ed as `bus_ctrl_perf_counter_t`) added.
  - **Note** enum `bus_ctrl_per_counter` no longer exists.
  - `enum clock_num_[rp2040|rp2350]` (typedef-ed as `clock_num_t`) added.
  - **Note** enum `clock_index` no longer exists.
  - `enum clock_dest_num_[rp2040|rp2350]` (typedef-ed as `clock_dest_num_t`) added.
  - `enum gpio_function_[rp2040|rp2350]` (typedef-ed as `gpio_function_t`) added.
  - **Note** enum `gpio_function` no longer exists.
- `hardware_timer`
- `NUM_TIMERS` has been renamed to `NUM_ALARMS` as that's what it was (4).

Build

- There are major CMake build changes. If you are maintaining your own non-CMake build, you will have to make extensive changes by looking at the differences yourself.
- All SDK headers are now "system" includes.
- You can now specify `rp2350-arm-s` (Arm Secure) and `rp2350-riscv` (RISC-V) as well as the previous `rp2040` (default) and `host`.
- Setting `PICO_BOARD=some_board` will now set `PICO_PLATFORM` if one is specified in `some_board.h` since most boards either use exclusively RP2040 or RP2350.
- `PICO_PLATFORM` also supports `rp2350` but this gets replaced with the value `PICO_DEFAULT_RP2350_PLATFORM` which you can set in your environment or `CMakeLists.txt`. Many of the boards for RP2350 - including `pico2`- select `rp2350` as the `PICO_BOARD` to honour your preference.

- PICO_PLATFORM, PICO_BOARD and other variables will be taken from your environment if not otherwise defined now retain their value after the first CMake invocation. i.e. a pre-existing CMake build configuration directory will not change based on your environment if you re-run cmake.
- PICO_BOARD=pico_w is no longer an odd child out requiring a CMake board file; support for CYW43 Wi-Fi can now be specified in the board header.
- ELF2UF2 is now replaced by use of picotool which will be built as part of your build if not installed on the system. See the picotool GitHub repository for more details on building and installing it locally.
- PICO_GCC_TRIPLE can now be a ';' separated list as well as a single value.
- NOTE: This release of the SDK does not support writing Arm Non-Secure binaries to run under the wing of an Arm Secure binary. This support will be added in a subsequent release.
- Compiler support is widening - we always recommend a recent version.)
- All recent GCCs are supported on Arm. (GCC 14 has not yet been tested for full support though).
  - Very recent GCCs are required on RISC-V due to the bleeding-edge nature of some of the processor instructions.
  - Recent LLVM Embedded Toolchain for ArmRM versions are supported on Arm.
  - Pigweed LLVM is supported for Arm.
  - For further details see the Raspberry Pi Pico-series C/C++ SDK book.
- Bazel may be used to build the SDK on Arm. See the README. Note that the Bazel build is community-provided and maintained.

Building Documentation

- The docs build target to build the HTML code documentation now builds a set of documentation peculiar to your particular PICO_PLATFORM setting.
- PICO_PLATFORM=combined-docs can be used (just for building docs) to build the combined documentation for both RP2040 and RP2350.

Fixed Issues

You can see a list of individual commits here, and a list of resolved issues here.

Note these only include public changes made since version 1.5.1. The majority of new code and collateral fixes for the previously unannounced RP2350 were developed and committed in private and delivered as a single "squashed" commit.

Release 2.1.0 (25 November 2024)

Adds support for Pico 2 W.

Board Support

The following boards have been added and may be specified via PICO_BOARD:

- adafruit_feather_rp2350
- datanoisetv_rp2350_dsp

- hellbender_0001
- machdyne_werkzeug
- pico2_w
- pimoroni_pico_plus2_w_rp2350
- sparkfun_thingplus_rp2350

The following board configurations have been modified:

- pimoroni_plasma2350 - corrected flash size, renamed SPICE to SPCE
- pimoroni_tiny2350 - corrected flash size

Notable Library Changes/Improvements

Clock dividers in general

A variety of methods which set clock dividers using an integer part and a fractional part, which might have been hardware_xxx_set_clkdiv_int_frac(uint16_t div_int, uint8_t div_frac) have been modified to hardware_xx_set_clkdiv_int_frac8(uint32_t div_int, uint8_t div_frac). This has been done for consistency and to make the APIs more resistant to hardware changes. The old APIs are preserved for backwards compatibility.

Previously, when converting from floating-point clock divider values to the fixed point use by the hardware, the floating-point value was rounded down. The new default (as configured by PICO_CLKDIV_ROUND_NEAREST) is to round to the nearest achievable value. This minor change in behavior was deemed better in general, which is why the default was changed. You may set PICO_CLKDIV_ROUND_NEAREST=0 to restore the previous behaviour by default (note that individual libraries have their own configuration values which can be used to change the behaviour on a per-library basis).

cmsis

- Fixed exception renaming for RP2350.

hardware_adc

- Added PICO_ADC_CLKDIV_ROUND_NEAREST for controlling rounding of floating-point clock dividers.

hardware_clocks

- Corrected spelling of PICO_CLOCK_AJDUST_PERI_CLOCK_WITH_SYS_CLOCK to PICO_CLOCK_ADJUST_PERI_CLOCK_WITH_SYS_CLOCK. The former is still supported.
- vco_calc.py now outputs SYS_CLK_HZ in the CMake output, which is required for clock_get_hz(clk_sys) to return the correct value.
- Renamed clock_gpio_init_int_frac() to clock_gpio_init_int_frac8() to clarify that it takes an 8-bit fraction; the old name is still supported.
- Added clock_gpio_init_int_frac16() to specify the fraction with 16-bit precision (RP2350 has 16 bits of precision). This method can still be called on RP2040 in which case the low 8-bits are ignored.
- Added PICO_CLOCK_GPIO_CLKDIV_ROUND_NEAREST for controlling rounding of floating-point clock dividers.

hardware_dma

- Fixed `dma_channel_cleanup()` to disable the channel with the new DMA IRQs added in RP2350.

hardware_exception

- Added missing Cortex-M33 exception numbers.

hardware_flash

- Prevented flash functions `flash_range_eraae()`, `flash_range_program()`, and `flash_do_cmd()` from trashing the user's CS1 QMI configuration on RP2350.
- Fixed issue with `flash_safe_execute` on FreeRTOS SMP.

hardware_i2c

- Added `i2c_write_burst_blocking` and `i2c_read_burst_blocking` to send/receive multiple bytes without intervening stops.
- Fixed rare hang during `i2c_read_blocking`.

hardware_interp

- Renamed `interp_add_accumulater()` to `interp_add_accumulator()`. The old incorrect spelling is still supported.

hardware_pio

- Added `pio_sm_set_pins64()`, `pio_sm_set_pins_with_mask64()` and `pio_sm_set_pindirs_with_mask64()` to allow setting of >32 pins.
- Much improved documentation of how GPIO numbers > 32 are handled.
- Fixed a bug in the use of a "jmp pin" > 32.
- Fixed implementation of `sm_config_set_in_pin_count()`.
- Renamed `sm_config_set_clkdiv_int_frac()` to `sm_config_set_clkdiv_int_frac8()` to clarify that it takes an 8-bit fraction; the old name is still supported. Note that "int" part in the new method is 32-bit not 16-bit for consistency with other `clkdiv` methods.
- Renamed `pio_calculate_clkdiv_from_float()` to `pio_calculate_clkdiv8_from_float()` to clarify that it produces an 8-bit fraction; the old name is still supported. Note that "int" part in the new method is 32-bit not 16-bit for consistency with other `clkdiv` methods.
- Added `PICO_PIO_CLKDIV_ROUND_NEAREST` for controlling rounding of floating-point clock dividers.

hardware_pwm

- Renamed `pwm_config_set_clkdiv_int_frac()` to `pwm_config_set_clkdiv_int_frac4()` to clarify that it takes an 4-bit fraction; the old name is still supported. Note that "int" part in the new method is 32-bit not 8-bit for consistency with other `clkdiv` methods.
- Added `PICO_PWM_CLKDIV_ROUND_NEAREST` for controlling rounding of floating-point clock dividers.

hardware_timer

- Fixed bug with alarms when using RP2350's new TIMER1.
- Corrected signature of hardware_alarm_get_irq_num() method added in SDK version 2.0.0. The variant that takes (and uses) a timer instance is called timer_hardware_alarm_get_irq_num().

pico_aon_timer

- Added aon_timer_start_calendar(), aon_timer_set_time_calendar(), aon_timer_get_time_calendar() and aon_timer_enable_alarm_calendar() methods. These are equivalent to the non-_calendar() variants, except they deal in calendar (date/) time, rather than time intervals. These new variants are preferred on RP2040 since otherwise a date/time conversion must be performed which pulls in a lot of C library code. For the same reason, the pre-existing variants are preferred on RP2350. This discrepancy results from the different hardware used for the AON timer on RP2040 and RP2350.

pico_atomic

- Fixed atomic use between core 0 and core 1.

pico_async_context

- Fixed possible HardFault in execute_sync() on FreeRTOS.

pico_binary_info

- bi_Xpins_with_names() macros now work correctly when pin numbers are not in order.

pico_bootrom

- Added rom_reset_usb_boot_extra() which supports an "activity" GPIO pin > 32 and GPIO pin inversion (active low).
- Bootrom methods that may write to flash are now protected with flash_safe_execute(). This affects rom_flash_op() and rom_explicit_buy().

pico_bootsel_via_double_reset

- Fixed implementation on RP2350. Note the RP2350 bootrom also provides this support if enabled via OTP, however this library can be used when that is not enabled.

pico_crt0

- __HeapLimit is now correctly set by the default linker scripts again.
- Fixed linker option -Wl,--print-memory-usage showing 100% RAM used.

pico_clib_interface

- Made some small improvements to picolibc integration.

pico_cyw43_driver

- Allow user configuration of Wi-Fi pins (including pin numbers >32) and SPI clock, including dynamic SPI clock

configuration at runtime.

- Updated `cye43_driver` to revision `cf924bb`.
- Renamed `cyw43_set_pio_clkdiv_int_frac()` to `cyw43_set_pio_clkdiv_int_frac8()` to clarify that it takes an 8-bit fraction; the old name is still supported. Note that "int" part in the new method is 32-bit not 16-bit for consistency with other `clkdiv` methods.
- Renamed `CYW43_PIO_CLOCK_DIV_FRAC8` to `CYW43_PIO_CLOCK_DIV_FRAC`. The old name is still supported.
- RISC-V is now supported.
- Added `PICO_BTSTACK_CYW43_MAX_HCI_PROCESS_LOOP_COUNT` configuration option, which can be used to prevent starvation in high frequency Bluetooth scenarios.

pico_flash

- Support serial flash with >8 byte unique id, using the last 8 bytes rather than the first.

pico_float

- Added optimized `add`/`sub`/`mul` implementations for Hazard3 for better floating point speed.

pico_malloc

- Fixed deadlock in `calloc()` and `realloc()` with `picolibc`.

pico_platform

- Added `pico_default_asm_volatile_goto()`.

pico_standard_binary_info

- Added back `boot_stage2` binary info (missing in SDK version 2.0.0).

pico_stdio_uart

- Fixed `stdio_flush()` when used with `stdio_uart_init_full()`.
- Fixed race condition in `stdio_set_chars_available_callback()`.

pico_stdio_usb

- Fixed Windows issue with the device not showing up if the reset interface is disabled.
- Added support for resetting to USB boot with an activity LED pin > 32 or with the LED active low (on RP2350).
- Added `PICO_STDIO_USB_RESET_BOOTSEL_FIXED_ACTIVITY_LED_ACTIVE_LOW` setting for RP2350.

pico_time

- Fixed race condition which could cause alarms to be lost.
- Fixed continuous wakeup in `best_effort_wfe_or_timeout()` on RP2350.

pico_util

- Added `datetime_to_tm()` and `tm_to_datetime()` for converting C library date/times to/from RP2040 RTC date/times.
- Added `pico_localtime_r()` and `pico_mktime()` for use by `pico_util` time conversion code. These methods cass the equivalent C library function, but are defined weakly so the user can provide their own.

TinyUSB

- Updated TinyUSB to 0.17.0.

New Libraries

boot_bootrom_headers

Split out the headers defining the bootrom interface - that might be used outside the SDK - from `pico_bootrom` which is focused on calling the bootrom from the SDK, and has non-trivial dependencies.

hardware_xip_cache

Provides XIP cache maintenance APIs:

- RP2040 support for cache invalidation.
- RP2350 support for cache invalidation/cleaning/pinning.

Miscellaneous

- Numerous documentation corrections/improvements.
- Various build warnings fixed in exotic compiler configurations.
- RP2350 A0/A1 silicon are no longer supported.

pioasm

- Fixed disassembly of `mov rx_fifo, ⋯` and `mov ⋯, rx_fifo` instructions/

Build

- Made build dependent on any signature files or embedded-partition-table JSON.
- Added back .hex file output (lost in SDK version 2.0.0).
- Made `PICO_FLASH_SIZE_BYTES` and `PICO_CYW43_SUPPORTED` if specified in CMake, correctly affect the compiled code.
- Various corrections to library dependencies.
- Added `PANIC` and `AUTO_INIT_MUTEX` options to `pico_minimize_runtime()`.
- Made `boot_stage2` build reproducible (same binary if no source changes).

Bazel Build

- Add support for building on Raspberry Pi OS.
- More CMake build configuration options supported.
- Preview support for Wi-Fi builds.

New Examples

This release adds the following examples to the `pico_examples` repository:

Example	Description
`binary_info/blink_any`	Uses bi_ptr variables to create a configurable blink binary - see the separate readme for mote details
`binary_info/hello_anything`	Uses bi_ptr variables to create a configurable hello_world binary - see the separate readme for more details
`i2c/slave_mem_i2c_burst`	i2c slave example where the slave implements a 256 byte memory. This version inefficiently writes each byte in a separate call to demonstrate read and write burst mode.
`pico_w/wifi/picow_blink_slow_clock`	Blinks the on-board LED (which is connected via the WiFi chip) with a slower system clock to show how to reconfigure communication with the WiFi chip at run time under those circumstances
`pico_w/wifi/picow_blink_fast_clock`	Blinks the on-board LED (which is connected via the WiFi chip) with a faster system clock to show how to reconfigure communication with the WiFi chip at build time under those circumstances
`pico_w/wifi/picow_http_client`	Demonstrates how to make http and https requests
`pico_w/wifi/picow_http_client_verify`	Demonstrates how to make a https request with server authentication
`pico_w/wifi/freertos/picow_freertos_http_client_sys`	Demonstrates how to make a https request in NO_SYS=0 (i.e. full FreeRTOS integration)
`universal/blink`	Same as the blink example, but universal.
`universal/nuke_universal`	Same as the flash/nuke example, but universal. On RP2350 runs as a packaged SRAM binary, so is written to flash and copied to SRAM by the bootloader

Release 2.1.1 (18 February 2025)

This is a minor release of the SDK with many bug fixes and documentation improvements, along with some new features.

Highlights are listed below, or you can see the full list of individual commits here, and the full list of resolved issues here. Board Support

The following boards have been added and may be specified via `PICO_BOARD`:

- `sparkfun_iotnode_lorawan_rp2350`
- `waveshare_pico_cam_a`

- waveshare_rp2040_ble
- waveshare_rp2040_eth
- waveshare_rp2040_geek
- waveshare_rp2040_matrix
- waveshare_rp2040_pizero
- waveshare_rp2040_power_management_hat_b
- waveshare_rp2040_tiny
- waveshare_rp2040_touch_lcd_1.28
- waveshare_rp2350_eth
- waveshare_rp2350_geek
- waveshare_rp2350_lcd_0.96
- waveshare_rp2350_lcd_1.28
- waveshare_rp2350_one
- waveshare_rp2350_plus_4mb
- waveshare_rp2350_plus_16mb
- waveshare_rp2350_tiny
- waveshare_rp2350_touch_lcd_1.28
- waveshare_rp2350_zero

The following board configurations have been modified:

- adafruit_feather_rp2350 - Increased the XOSC startup delay
- seeed_xiao_rp2350 - Increased the default SPI clock divider
- waveshare_rp2040_lcd_0.96 - Renamed WAVESHARE_RP2040_LCD_ constants to WAVESHARE_LCD_
- waveshare_rp2040_lcd_1.28 - Renamed WAVESHARE_RP2040_LCD_ constants to WAVESHARE_LCD_

NOTE

This release changes the default PICO_XOSC_STARTUP_DELAY_MULTIPLIER from 1 to 6, unless specified by a board header file. This creates a delay of 6ms, since testing of the recommended crystal shows it can take up to this long to stabilize.

200 MHz Clock Support for RP2040

RP2040 has now been certified to run at a system clock of 200 MHz when using a regulator voltage of at least 1.15 volts.

The SDK by default performs clock setup for you before your program enters main(). If you haven't customized the clock configuration in any way, it will attempt to configure the system clock based on the value of SYS_CLK_MHZ (or SYS_CLK_KHZ /SYS_CLK_HZ if specified instead). Without further information from you, it can only do this for specific clock frequencies.

In prior versions of the SDK, only one specific clock frequency was defined per platform, 125 MHz for RP2040 and 150 MHz for RP2350, which also happen to be the default values for SYS_CLK_MHZ.

With this version of the SDK, you can now select a 200 MHz clock for RP2040 by setting SYS_CLK_MHZ=200 via preprocessor define. The regulator voltage will automatically be raised for you if necessary.

We may certify new frequencies for the different platforms in the future. The original SYS_CLK_MHZ defaults are left

unchanged because not all programs would function correctly at a different system clock frequency. If, however, your project would always benefit from the fastest clock, you may now define `PICO_USE_FASTEST_SUPPORTED_CLOCK=1` via CMake variable or as a preprocessor define, and it will always use the fastest supported system clock frequency for the platform in the future.

Notable Library Changes/Improvements

hardware_clocks

- Corrected documentation and implementation of `clock_configure()` supporting the full range of clock dividers.
- Added `PICO_USE_FASTEST_SUPPORTED_CLOCK` and PLL configuration for 200 MHz on RP2040.

hardware_flash

- Moved internal flash helper function to run from RAM instead of flash. This adds support to builds other than `COPY_TO_RAM`.

hardware_irq

- Added significantly improved documentation around IRQ handlers when using both cores.
- Added `enable_interrupts()` and `disable_interrupts()` methods for when you don't care about saving or restoring the current interrupt state.
- Added `irq_has_handler()` method to tell if a handler is installed for a particular IRQ number.

hardware_pio

- Fixed support for WAIT gpio with GPIO number >= 32.

pico_aon_timer

- Added a 2 RTC-clock propagation delay at the end of `aon_timer_set_time_calendar()` on RP2040, such that reading back the time immediately afterwards returns the correct value.

pico_bootrom

- Added `rom_data_lookup_inline()` to complement `rom_data_lookup()`.

pico_btstack

- Updated BTStack to 1.6.2 from 1.6.1.
- Updated Raspberry Pi BTStack license to cover Pico 2 W, Pico 2 WH, and RM2.

pico_cyw43_driver

- Updated cyw43_driver to revision `c1075d4b`.
- Fixed rare issue when loading firmware.

pico_double

- Significantly cleaned up and improved documentation.
- Implemented the full complement of double conversion functions defined in `pico/double.h` across both RP2040 and RP2350 variants of `pico_double_pico`.

pico_float

- Significantly cleaned up and improved documentation.
- RP2350 `pico_float_pico_dcp` variant now enables `-msoft-float`, since if you've chosen to use DCP instead of VFP for single-precision floating-point, you probably don't want the compiler emitting inline VFP instructions either.
- Implemented the full complement of float conversion functions defined in `pico/float.h` across RP2040 and all RP2350 variants of `pico_float_pico`.

pico_flash

- Fixed a build error when using FreeRTOS with config `SUPPORT_DYNAMIC_ALLOCATION=0`.

pico_lwip

- Fixed build with `PPP_SUPPORT=1` when using `pico_lwip_nosys`.

pico_mbedtls

- Added correct cleanup of RP2350 SHA256 state during `mbedtls_sha256_free()`.

pico_multicore

- Added `multicore_lockout_victim_deinit()`
- `multicore_reset_core1()` now marks Core 1 as de-initialized w.r.t. `multicore_lockout_victim_` functions, allowing `multicore_lockout_victim_init()` to perform correctly after the reset.

pico_runtime_init

- Added `SYS_CLK_VREG_VOLTAGE_AUTO_ADJUST` to indicate the voltage regular should be set to `SYS_CLK_VREG_VOLTAGE_MIN` during default clock setup in order to support the configured system clock frequency.

pico_sha256

- Added `pico_sha256_cleanup()` to clean up from an in-progress SHA256 operation which was not completed via `pico_sha256_finish()`.

pico_stdio_usb

- Allow user to override `CFG_TUD_CDC_RX_BUFSIZE`, `CFG_TUD_CDC_TX_BUFSIZE`, and `CFG_TUD_CDC_EP_BUFSIZE` defines to increase performance.

pico_time

- Fixed a rare race condition that could cause alarms/repeating timers to get "lost".

TinyUSB

- Updated TinyUSB to 0.18.0 from 0.17.0

FreeRTOS

- Upstreamed FreeRTOS support for RP2350 (Arm/RISC-V) to https://github.com/FreeRTOS/FreeRTOS-Kernel. However, this is not yet in any official release, so you should use the latest from the main branch there. Be sure to initialize the submodules, since RP2350 support is in a submodule.
- If your project embeds FreeRTOS_Kernel_import.cmake, you should update to the latest version here which works for both RP2040 and RP2350.

Pioasm

- Fixed encoding of WAIT GPIO with GPIO number >= 32.
- Python output now correctly emits word(x) for all PIO version 1 (RP2350) PIO instructions.

SVD

- Fixed access type for DMA `CHAN_ABORT` register to be read-write (with clear-on-write) for both RP2040 and RP2350.

Build

- Added support for GCC 14.
- Added support for LLVM Embedded Toolchain For Arm 19.x.
- Added support for multiple `.pio` files in `pico_generate_pio_header()`.
- Updated `.DIS` files for builds using LLVM/Clang on RP2350 to contain correct disassembly for VFP floating point instructions.
- Fixed some newer CMake version deprecation warnings.
- Added explicit license to pico_sdk_import.cmake as it is copied into external projects.

Bazel Build

- Updated LLVM/Clang toolchain to fix stack overflow issue with `fma()`-related math functions.

New Examples

This release adds the following examples to the `pico_examples` repository:

Example	Description
`pico_w/wifi/mqtt/picow_mqtt_client`	Demonstrates how to implement a MQTT client application
`pio/uart_pio_dma`	Send and receive data from a UART implemented using the PIO and DMA

Example	Description
usb/device/dev_multi_cdc	A USB CDC device example with two serial ports, one of which is used for standard SDK stdio. The example exposes two serial ports over USB to the host. The first port is used for stdio, and the second port is used for a simple echo loopback. You can connect to the second port and send some characters, and they will be echoed back on the first port while you will receive a "OK\r\n" message on the second port indicating that the data was received.